

CONNECT360 - AN INTELLIGENT AND SECURE PLATFORM FOR SKILLED LABOUR BOOKING

PHASE I REPORT

Submitted by

DHARSHINI R S – 230701076

ENIYA B A – 230701085

JAYADHARSINI M – 230701127

JAYAPRADHA P – 230701130

in partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

IN

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE, CHENNAI

ANNA UNIVERSITY: CHENNAI 600 025

September 2026

ANNA UNIVERSITY, CHENNAI

BONAFIDE CERTIFICATE

Certified that this Report titled "**CONNECT360- AN INTELLIGENT AND SECURE PLATFORM FOR SKILLED LABOUR BOOKING**" is the bonafide work of **DHARSHINI R S (230701076), ENIYA B A (230701085), JAYADHARSINI M (230701127), JAYAPRADHA P (230701130)** who carried out the work under my supervision. Certified further that to the best of my knowledge the work reported herein does not form part of any other thesis or dissertation on the basis of which a degree or award was conferred on an earlier occasion on this or any other candidates.

This project addresses the following Sustainable Development Goal(s): Goal 8 – Decent Work and Economic Growth, Goal 9 – Industry, Innovation and Infrastructure.

SIGNATURE

Dr. Manoranjini M.E., Ph.D.,
Professor and Head,
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

SIGNATURE

MS. Divya M M.E.,
Assistant Professor
Department of Computer Science
and Engineering,
Rajalakshmi Engineering College,
Chennai – 602 105.

Submitted to Project Viva-Voce Examination held on _____

Internal Examiner

External Examiner

ABSTRACT

The rapid growth of urban areas has increased the need for reliable and on-demand home technical services such as electrical work, plumbing, HVAC repair, carpentry, painting, and appliance maintenance. However, existing digital service marketplaces may face challenges such as delayed technician assignment, duplicate bookings, limited transparency in worker selection, exposure of personal information, and communication outside the platform. To address these challenges, this project presents Connect360, a cloud-based, serverless marketplace designed to connect customers with skilled service professionals through an integrated digital platform.

Connect360 is developed using a responsive React-based frontend and a serverless backend built with Amazon Web Services (AWS). The system uses AWS Cognito, Lambda, API Gateway, and DynamoDB to support authentication, service operations, and data management. Its worker matching system considers service match, location, ratings, verified experience, availability, and reliability to identify suitable professionals. A Priority Booking feature supports urgent requests with server-side availability checks and concurrency-control mechanisms designed to prevent duplicate worker assignment in the tested scenarios.

The platform also supports worker verification, booking lifecycle tracking, and role-based access for customers, workers, and administrators. Location-based distance calculations and road-routing services using Leaflet, OSRM, and Nominatim support worker matching and service tracking, while S3 storage is used for verification documents. Authorized customer-worker calling is restricted to active bookings. The serverless architecture uses Terraform with S3 and CloudFront for frontend hosting, providing a structured cloud environment for the Connect360 Phase-I implementation.

ACKNOWLEDGEMENT

Initially we thank the Almighty for being with us through every walk of our life and showering his blessings through the endeavor to put forth this report. Our sincere thanks to our Chairman **Mr. S. MEGANATHAN, B.E, F.I.E.,** our Vice Chairman **Mr. ABHAY SHANKAR MEGANATHAN, B.E., M.S.,** and our respected Chairperson **Dr. (Mrs.) THANGAM MEGANATHAN, Ph.D.,** for providing us with the requisite infrastructure and sincere endeavoring in educating us in their premier institution.

Our sincere thanks to **Dr. S.N. MURUGESAN, M.E., Ph.D.,** our beloved Principal for his kind support and facilities provided to complete our work in time. We express our sincere thanks to **Dr. J. Manoranjini, Ph.D.,** Professor and Head of the Department of Computer Science and Engineering for her guidance and encouragement throughout the project work. We convey our sincere and deepest gratitude to our internal guide, **Ms. Divya M, M.E.,** Assistant Professor, Department of Computer Science and Engineering for her valuable guidance throughout the course of the project. We thank our Project Coordinator, **Mr. Karthik, B.E., M.E.,** Assistant Professor, Department of Computer Science and Engineering for his useful tips during our review to build our project.

DHARSHINI R S– 230701076

ENIYA B A– 230701085

JAYADHARSINI M– 230701127

JAYAPRADHA P – 230701130

TABLE OF CONTENTS

ABSTRACT	iii
ACKNOWLEDGEMENT	iv
LIST OF TABLES	viii
LIST OF FIGURES	ix
LIST OF ABBREVIATIONS	xi
1 INTRODUCTION	1
1.1 GENERAL	1
1.2 OBJECTIVE	2
1.3 EXISTING SYSTEM	3
1.4 PROPOSED SYSTEM	4
2 LITERATURE SURVEY	6
2.1 LITERATURE REVIEW	6
2.2 COMPARISON AND DISCUSSION	9
2.3 CONCLUSION	10
3 SYSTEM DESIGN	11
3.1 SYSTEM ARCHITECTURE	11
3.2 SYSTEM REQUIREMENTS	13
3.2.1 SOFTWARE REQUIREMENTS	13
3.2.2 HARDWARE REQUIREMENTS	13
3.2.3 DATABASE / DATA REQUIREMENTS	14
3.2.4 DEPLOYMENT AND SCALING	14
3.3 SYSTEM DESIGN DIAGRAMS	15

3.3.1	USE CASE DIAGRAM	15
3.3.2	ACTIVITY DIAGRAM	16
3.3.3	DATA FLOW DIAGRAM (LEVEL 0 & LEVEL 1)	18
3.3.4	SEQUENCE DIAGRAM	20
4	PROJECT DESCRIPTION	21
4.1	METHODOLOGIES	21
4.2	MODULES	22
4.2.1	MODULE 1: AUTHENTICATION AND RBAC CONTROL	22
4.2.2	MODULE 2: WORKER PROFILE & GEOSPATIAL REGISTRATION	23
4.2.3	MODULE 3: INTELLIGENT MATCHING ENGINE	23
4.2.4	MODULE 4: TRANSACTIONAL PRIORITY DISPATCH	25
4.2.5	MODULE 5: HYBRID SPATIAL ROUTING & POLLING-BASED TRACKING	26
4.2.6	MODULE 6: CONVERSATIONAL AI ASSISTANT & CONTACT-INFORMATION	26
4.2.7	MODULE 7: CONTROLLED CUSTOMER-WORKER CALLING	27
5	IMPLEMENTATION AND RESULT DISCUSSION	28
5.1	IMPLEMENTATION RESULTS	28

5.2 EVALUATION AND PERFORMANCE ANALYSIS	32
5.2.1 METRIC 1: MATCHING ALGORITHM	32
EXECUTION LATENCY	
5.2.2 METRIC 2: MATCHING ENGINE RANK	35
SENSITIVITY & INVERSION	
5.2.3 METRIC 3: TRANSACTIONAL CONCURRENCY	38
& RACE PREVENTION	
5.2.4 METRIC 4: ROAD DISTANCE VS.	41
HAVERSINE DISPARITY	
5.2.5 METRIC 5: CONTACT-INFORMATION	45
REDACTION EVALUATION	
5.2.6 METRIC 6: STRUCTURED OUTPUT JSON	48
SCHEMA CONFORMITY & URGENCY	
CLASSIFICATION	
5.3 OVERALL RESULT DISCUSSION	51
6 CONCLUSION AND FUTURE WORK	52
6.1 CONCLUSION	52
6.2 FUTURE WORK	54
REFERENCES	55

LIST OF TABLES

TABLE NO.	TABLE NAME	PAGE NO.
2.1	Feature-Based Comparison of Service Marketplace Approaches	10
3.1	Software Requirements of the Proposed Connect360 System	13
3.2	Hardware Requirements of the Proposed Connect360 System	13
3.3	Amazon DynamoDB Single-Table Schema Entities & Index Design	14
4.1	Multi-Criteria Weighting Configurations for Normal vs. Priority Booking	25
5.1	Matching Algorithm Execution Latency Across Candidate Pool Sizes	33
5.2	Rank Inversion & Sensitivity Analysis Under Weight Reallocation	37
5.3	Transactional Concurrency & Conflict Abort Latency Across Thread Loads	40
5.4	Haversine vs. OSRM Road Distance Disparity Across Distance Tiers	43
5.5	Contact-Information Redaction Performance Across 10 Categories	46
5.6	Structured Output Schema Conformity & Urgency Classification Across Dimensions	49

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	Connect360 Three-Layer Cloud & Serverless System Architecture	11
3.2	Connect360 Use Case Diagram	16
3.3	Connect360 Priority Booking Activity Diagram	17
3.4	Connect360 Data Flow Diagram (DFD Level 0 - Context Diagram)	19
3.5	Connect360 Data Flow Diagram (DFD Level 1 - Modular Decomposition)	19
3.6	Sequence Diagram: Priority Booking Atomic Acceptance & Commit	20
5.1	User Login Interface	29
5.2	Customer Dashboard	29
5.3	Find Professionals Interface	29
5.4	Customer Booking History	30
5.5	Priority Booking Request	30
5.6	Customer Priority Booking Tracking	30
5.7	Worker Dashboard	31
5.8	Worker Bookings Management	31
5.9	Worker Verification Status	31
5.10	Priority Dispatch Radar and Polling-Based Tracking	32
5.11	Matching Algorithm Execution Latency Vs. Candidate Pool Size	34
5.12	Matching Engine Rank Sensitivity & Inversion Analysis	37

FIGURE NO.	FIGURE NAME	PAGE NO.
5.13	Transactional Concurrency & Race Condition Prevention	40
5.14	Road Distance Vs. Haversine Disparity & Tortuosity Factor Distribution	43
5.15	Connect360 Contact-Information Redaction Confusion Matrix	46
5.16	Structured Output JSON Schema Conformity & Urgency Classification	49

LIST OF ABBREVIATIONS

ABBREVIATION	DESCRIPTION
AI	Artificial Intelligence
API	Application Programming Interface
AWS	Amazon Web Services
DFD	Data Flow Diagram
ETA	Estimated Time of Arrival
F1	F1-Score
FN	False Negative
FP	False Positive
GSI	Global Secondary Index
HTTP	Hypertext Transfer Protocol
HTTPS	Hypertext Transfer Protocol Secure
HVAC	Heating, Ventilation, and Air Conditioning
JSON	JavaScript Object Notation
JWT	JSON Web Token
ML	Machine Learning
NoSQL	Not Only SQL
OSRM	Open Source Routing Machine
P90	90th Percentile
P99	99th Percentile
PK	Partition Key
PII	Personally Identifiable Information
RBAC	Role-Based Access Control
REST	Representational State Transfer
RL	Reinforcement Learning
S3	Simple Storage Service
SK	Sort Key

ABBREVIATION	DESCRIPTION
TN	True Negative
TP	True Positive
URL	Uniform Resource Locator

CHAPTER 1

INTRODUCTION

1.1 GENERAL

The rise of urban households and busy lifestyles has created a demand for reliable on-demand skilled labour. Home services typically encompass electrical, plumbing, carpentry, cleaning, equipment maintenance, and other household-related assistance. Customers traditionally identify professionals through personal contacts, local references, manual searches, or informal networks, which offer limited support in terms of comparing skills, experience, availability, ratings, reliability, trust, and distance. Skilled informal workers may find themselves limited by a lack of digital presence to advertise their services and receive requests. A digital marketplace can therefore help to enhance worker discovery while providing professionals with visibility and access to customers.

Digital service platforms have attempted to address this fragmented environment by bringing workers together through centralized worker discovery and booking. However, proximity-based matching may not necessarily identify a suitable professional. A nearby worker may lack the required skills, experience, or availability. Normal and urgent requests also have different requirements. While planned services can take into consideration service match, trust, experience, availability, reliability, and distance, urgent requests must place significantly more emphasis on the availability of workers who can reach the location suitably. The recommendation of a worker must therefore take into consideration the requirements of each booking rather than proximity.

Home-service marketplaces can also be plagued by verification, booking management, data security, and concurrent worker allocation. Customers require confidence in the identification and credentials of a professional, while workers need facilities for managing their profiles, services, availability, bookings, and reviews. During concurrent requests, the booking conflict arises unless the worker's availability and acceptance are protected by server-side checks. Connect360 addresses these requirements with Amazon Cognito for authentication and roles, access-controlled Amazon S3 storage for verification documents, DynamoDB for application data, and conditional and transactional operations for booking consistency.

Connect360 is implemented as a serverless skilled home-services marketplace integrating worker discovery, verification, multi-criteria matching, booking, priority services, and administration. The Candidate Matching Engine ranks eligible professionals using a deterministic weighted scoring approach based on service match, trust, experience, availability, reliability, and distance. Normal and Priority Booking use different matching priorities. Leaflet and OSRM are used to support map-based service tracking. The platform is implemented using React, Vite, AWS Cognito, API Gateway, Lambda, DynamoDB, S3, CloudFront, and Terraform.

1.2 OBJECTIVE

The goal of the project involved designing, implementing, and evaluating a hyperlocal skilled-labour service marketplace that supported transparent worker selection, secure booking, and controlled communication. Connect360 addressed the following:

- Design of a multi-criteria matching engine with an $O(N)$ candidate-scoring stage that considered service match, trust,

experience, availability, reliability, and distance for Normal and Priority Booking.

- Prevention of booking conflicts using Amazon DynamoDB conditional and transactional operations to protect worker availability and prevent duplicate assignment during concurrent requests.
- Implementation of secure worker verification and data protection through controlled document handling, role-based access, and secure application data management.
- Development of a hybrid spatial routing framework using Haversine distance for candidate ranking and OSRM road routing for service tracking.
- Support for controlled customer-professional communication with calling restricted to active bookings and authorized participants.
- Evaluation of the system using matching latency, ranking behaviour, booking concurrency, spatial distance differences, and other performance measures.

1.3 EXISTING SYSTEM

Existing approaches to on-demand home-service discovery may face a number of limitations:

- Centralized and opaque worker selection: Proximity-based or centralized selection may not transparently consider service skills, experience, ratings, reliability, and availability.
- Concurrency issues: Multiple users or workers may update the same booking or availability information simultaneously. Without server-side conditional updates and transaction control, conflicting booking states may arise.

- Limited verification and data protection: Basic service platforms may not fully integrate professional verification and application-level protection for customer, worker, and verification information.
- Computational considerations in spatial routing: Straight-line distance is efficient but does not represent road distance, while calculating complete road routes for every candidate can increase computational and network overhead.
- Direct telephony and limited verification: Traditional service directories may expose contact information directly and provide limited mechanisms for verifying professional identity, credentials, availability, and reviews.

1.4 PROPOSED SYSTEM

The proposed architecture for Connect360 addresses the limitations of existing home-service systems by presenting an integrated serverless cloud solution. The feature set includes the following:

- Dual-mode intelligent worker matching engine: The Candidate Matching Engine evaluates eligible workers based on service match, trust, experience, availability, reliability, and distance. In Normal Booking, the matching score prioritizes service suitability and worker trust, whereas Priority Booking prioritizes availability and distance to identify an appropriate available professional for the request. The system therefore applies different scoring priorities based on the booking type.
- Transaction concurrency protection: Within the Priority Dispatch Handler, the system utilizes Amazon DynamoDB conditional operations and transactional mechanisms to protect worker availability and booking acceptance during concurrent

operations. After a worker is selected for a priority request, the system checks whether the booking remains available and whether the professional is still eligible before confirming the change on the server side. This mechanism is designed to prevent duplicate worker assignment during concurrent booking operations.

- Secure worker verification and access control: Connect360 provides a structured worker verification process in which professionals can submit required identification and supporting documents through controlled application workflows. Verification documents are stored using secure S3 access mechanisms, while authentication and role-based authorization restrict access to customer, worker, and administrator functions. This supports controlled handling of professional information throughout the platform.
- Hybrid spatial routing engine: Connect360 uses a hybrid spatial strategy for worker identification and service tracking. Haversine distance is used for candidate ranking, while Leaflet and OSRM provide map rendering and road-based routing with distance and driving-time estimates. Nominatim resolves addresses into geographic coordinates, and routes can be recalculated when the configured movement threshold is exceeded.
- Role-restricted customer-worker calling: Connect360 uses the CallButton component to provide direct calling functionality, with the call button available only for active bookings involving the relevant customer and professional. This enables communication within an active booking, while role-based restrictions control booking-related operations and privacy-safe handling of specified contact information.

CHAPTER 2

LITERATURE SURVEY

2.1 LITERATURE REVIEW

The development of the Connect360 solution is inspired by research into local service-provider recommendation, skill-based matching, personalized and trust-aware recommendation, spatial worker allocation, priority-based task assignment, artificial intelligence, and cloud-based service systems. These research areas form the foundation for the different components of the proposed home-service platform.

Local Service-Provider Recommendation and Skill-Based Matching

Ezhilnilavan et al. [1] propose a machine-learning-based local community service-provider recommendation system that considers provider information, verification, user preferences, and reviews to recommend suitable local service providers. The study highlights the importance of combining provider information with user preferences but does not cover worker availability, service urgency, booking management, and service execution.

Munir et al. [13] investigate developer recommendation and team formation in collaborative crowdsourcing platforms by representing developer skills and task requirements for matching. Their work demonstrates the importance of representing worker capabilities when matching workers with task requirements, but focuses on developer and team recommendation instead of home-service workers where location, availability, trust, and booking conditions are also important.

Kumar et al. [19] propose a skill-based rule-matching approach for job recommendation using worker qualifications and skills. The study demonstrates how explicit skills and qualifications can identify suitable

candidates, but does not consider the broader combination of availability, reliability, trust, geographic proximity, and service urgency required for home-service allocation.

Personalized, Trust-Aware Recommendation and Artificial Intelligence

Sun et al. [2] propose a weighted preference optimization approach based on knowledge graphs and large language models for personalized service recommendation. The approach combines structured information with language-model processing to capture user preferences, but focuses mainly on personalized recommendation rather than complete worker allocation and booking.

Ge et al. [6] develop LLMGCL which combines graph contrastive learning with large language models for recommendation. The study demonstrates how graph relationships and language-model information can improve recommendation, but does not specifically address worker availability, geographic proximity, booking status, or service urgency.

Zhao et al. [14] examine recommender systems in the era of large language models while Zheng et al. [15] investigate the integration of collaborative semantics with large language models for recommendation. These studies demonstrate the potential of LLMs for understanding user requirements and supporting recommendation, but do not specifically integrate natural-language service requests with worker ranking, booking management, and service execution.

Khelloufi et al. [11] propose a hybrid feature and trust-aggregation recommender system that incorporates trust information into recommendation. The study highlights the importance of trust in recommendation decisions, but does not specifically combine worker

verification, booking history, availability, and geographic location within a home-service marketplace.

Chen et al. [12] investigate reliable personalized recommendation under uncertain missing modalities and highlight the importance of reliability when complete information is unavailable. However, the study does not specifically combine worker trust, service capability, availability, and geographic proximity for home-service worker selection.

Spatial Worker Allocation and Priority-Based Dispatch

Spatial information is important in home-service applications because service professionals need to travel to the customer's location. Zhao et al. [3] propose a heterogeneous graph reinforcement-learning approach for dependency-aware multi-task allocation in spatial crowdsourcing. The study demonstrates how worker-task relationships and spatial information can support allocation decisions, but does not specifically combine spatial allocation with worker skills, trust, experience, and availability for home services.

Wu et al. [5] investigate adaptive task assignment using a human-in-the-loop approach, demonstrating the importance of adaptive assignment in spatial task environments. However, the approach does not specifically address the complete home-service booking lifecycle or multi-factor worker ranking.

Deng et al. [7] explore task recommendation in spatial crowdsourcing by considering diversity and coverage. The study demonstrates that spatial task recommendation can consider factors beyond distance, but focuses on spatial crowdsourcing rather than skilled home-service worker selection.

Wu et al. [8] develop a prediction-aware adaptive task-assignment approach for spatial crowdsourcing while Zhao et al. [9] propose an efficient geographic partition framework for spatial task allocation. These approaches address

spatial allocation but do not directly integrate service capability, worker trust, availability, and booking requirements.

Ma et al. [10] propose a clustering-based priority-queue approach for spatial task assignment by considering task priorities. The work demonstrates how priority can influence allocation decisions, but does not specifically combine service matching, worker trust, availability, proximity, and booking state for home-service Priority Booking.

Chen et al. [4] investigate fairness-aware task recommendation and assignment in spatial crowdsourcing. The study highlights the importance of fairness in task allocation, but does not specifically address the urgency and worker-selection requirements of a skilled home-service platform.

Cloud-Based and Serverless Service Systems

Cloud-native and serverless technologies provide the computational infrastructure required for distributed service applications. Refonaa et al. [16] examine serverless architecture for cloud automation while Ekwe-Ekwe and Amos [17] analyze public Function-as-a-Service providers and their characteristics. Adeppady et al. [18] investigate resource management for serverless services while Pasam et al. [20] explore scalable AI pipelines using cloud-native technologies for intelligent transactional systems. These studies demonstrate the suitability of serverless and cloud-native technologies for distributed applications, but primarily focus on infrastructure and resource management rather than integrating worker discovery, matching, booking, and service execution.

2.2 COMPARISON AND DISCUSSION

To contextualize Connect360's architectural contributions, Table 2.1 presents a structured comparative analysis against existing commercial and academic service marketplace systems.

Table 2.1: Feature-Based Comparison of Service Marketplace Approaches

Evaluation Dimension	Urban Company	TaskRabbit	Traditional Relational Dispatch	Connect360 (Proposed)
Matching Heuristic	Platform-based service discovery and worker matching.	Profile-based tasker discovery and selection.	Rule-based or nearest-available dispatch.	Transparent dual-mode weighted multi-criteria ranking.
Dispatch Latency	Not evaluated in this study.	Not evaluated in this study.	Not evaluated in this study.	Approximately 0.07 ms at 100 candidates (measured benchmark).
Concurrency Control	Platform-managed booking system.	Platform-managed task booking.	Relational transaction or locking mechanisms.	DynamoDB conditional and transactional writes; no double-booking observed under tested conditions.
Spatial Routing	Location-based service discovery.	Location-based tasker discovery.	Geographic proximity-based dispatch.	Hybrid Haversine ranking with OSRM road routing.
AI Assistance & Triage	Not evaluated in this study.	Not evaluated in this study.	Rule- or keyword-based processing.	Gemini-based conversational triage with structured JSON and rule-based fallback.
PII Redaction Defense	Not evaluated in this study.	Not evaluated in this study.	Not evaluated in this study.	Deterministic post-generation contact-detail filtering (99.20% recall on the evaluated test set).
Telephony Privacy	Platform-mediated communication.	Platform-mediated communication.	Direct phone communication.	Native-device calling restricted to active bookings with role-based access.

2.3 CONCLUSION

The literature survey illustrates that current research has explored key areas such as skill-based matching, personalized and trust-aware recommendation, spatial task allocation, task priority, fairness, artificial intelligence, and cloud-based systems. Yet, the surveyed articles do not combine these requirements in a skilled home-service workflow.

Connect360 expands upon these research directions by incorporating multi-criteria worker ranking, worker verification, transactional booking control, Priority Booking, conversational AI assistance, and spatial service support in a serverless home-service marketplace.

CHAPTER 3

SYSTEM DESIGN

3.1 SYSTEM ARCHITECTURE

The system architecture of Connect360 is a decoupled cloud-based three-layer structure, which comprises the Client Layer, Serverless Compute Layer, and Data & External Services Layer, as shown in Figure 3.1. The architecture partitions the process of user-facing application, serverless business logic, and data and external service integration, thus enabling the main functions of the application to be grouped into focused components and services.

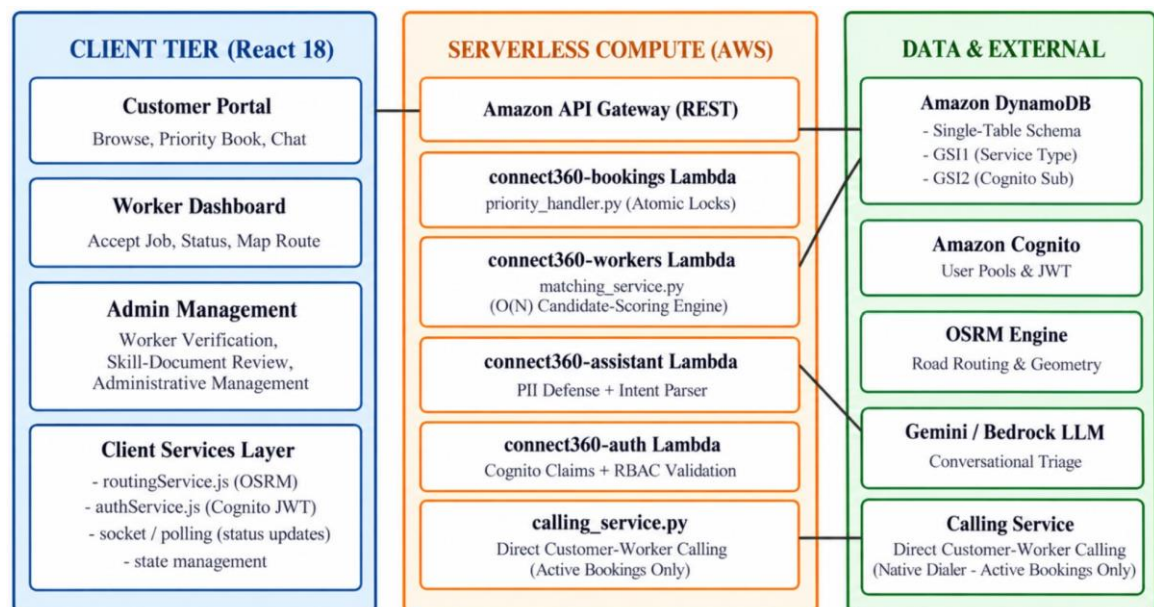


Figure 3.1: Connect360 Three-Layer Cloud & Serverless System Architecture

The Client Tier is implemented in React 18 and provides role-based interfaces for Customers, Workers, and Administrators. The Customer Portal supports worker browsing, Priority Booking, and conversational interaction. The Worker Dashboard enables workers to accept jobs, update booking status, and view the service routing. The Admin Management interface controls worker

verification, skill-document review, and administrative management. The Client Services Layer incorporates supporting functions such as the OSRM-based routing via `routingService.js`, Amazon Cognito JWT authentication via `authService.js`, syncing of booking and status information with polling, and client-side state handling.

The Serverless Compute Layer is implemented with AWS Lambda functions with Amazon API Gateway as the REST API entry point. The `connect360-bookings` Lambda supports booking operations and Priority Booking via `priority_handler.py`, including atomic locking mechanisms. The `connect360-workers` Lambda incorporates the `matching_service.py` Candidate Matching Engine, which implements the candidate-scoring $O(N)$ stage for worker prioritization. The `connect360-assistant` Lambda provides PII defense and intent parsing for conversational requests. The `connect360-auth` Lambda handles Amazon Cognito claims and Role-Based Access Control (RBAC) validations. The `calling_service.py` component supports direct customer-worker calling for active bookings through the device's native dialer.

The Data & External Services Tier provides persistent storage, authentication, routing, conversational AI, and calling-related services. Amazon DynamoDB is implemented as a single-table schema with two Global Secondary Indexes, GSI1 for Service Type and GSI2 for Cognito Sub. Amazon Cognito provides the user pools and the JWT-based authentication information. The OSRM Engine provides the road routing and route geometry for spatial operations, while the conversational assistant uses Google Gemini for conversational triage and intent-related processing. The Calling Service supports direct customer-worker communications through the device's native dialer, with calling access restricted to active bookings.

3.2 SYSTEM REQUIREMENTS

3.2.1 Software Requirements

Table 3.1: Software Requirements of the Proposed Connect360 System

Component	Technology / Specification	Version / Details
Frontend Framework	React.js (SPA) with Vite build tool	React 18.3+, Vite 5.0+
Programming Languages	JavaScript (ES2022+), Python	Node.js 20.x, Python 3.12
Cloud Serverless Compute	AWS Lambda with API Gateway	Python 3.12 runtime, 512 MB memory
NoSQL Database	Amazon DynamoDB	On-Demand Capacity, Single-Table Schema
Authentication Provider	Amazon Cognito User Pools	OAuth 2.0 / OpenID Connect, JWT Claims
Object Storage	Amazon S3	Worker verification document storage
Content Delivery	Amazon CloudFront	Frontend content distribution
DNS Service	Amazon Route 53	DNS management
Secrets Management	AWS Secrets Manager	API key storage
Spatial Routing Engine	Open Source Routing Machine (OSRM)	Road routing and distance information
Map Rendering	Leaflet	Map rendering and visualization
Geocoding Service	Nominatim	Address-to-coordinate conversion
AI / Foundation Model	Google Gemini API	Structured conversational AI output
Infrastructure-as-Code	Terraform	Cloud infrastructure deployment
Telephony	Device Native Dialer	Authorized calling for active bookings
Testing & Evaluation	Pytest, NumPy	Performance and functional evaluation

3.2.2 Hardware Requirements

Table 3.2: Hardware Requirements of the Proposed Connect360 System

Environment	Parameter	Minimum Specification
Client Devices (Customer/Worker)	Device Support	Responsive mobile-first web application
	Processor	Not specified in README
	Memory (RAM)	Not specified in README
	Network Connectivity	Internet connection required; deployed frontend/API use HTTPS
Serverless Cloud Compute	Runtime	AWS Lambda — Python 3.11
	Lambda Functions	7 domain Lambda functions
	Database Storage	Amazon DynamoDB — Single-Table Design, 2 GSIs, PAY_PER_REQUEST
Object Storage	Verification Storage	Amazon S3 — private, encrypted, pre-signed URLs
Maps & Routing	Mapping / Routing	Leaflet.js + OSRM; Nominatim geocoding; Google Maps Routes API fallback
Benchmark Environment	CPU, RAM, OS	Not specified in README

3.2.3 Database / Data Requirements

Connect360 utilizes the Amazon DynamoDB single-table design, which organizes related entities according to application access patterns and reduces the need for relational-style joins in operational queries. Users, Workers, Bookings, Reviews, and Activity logs are stored in the connect360-main-dev table using composite primary keys (Partition Key PK and Sort Key SK) and two Global Secondary Indexes (GSI1 and GSI2) as seen in the Table 3.3 below.

Table 3.3: Amazon DynamoDB Single-Table Schema Entities & Index Design

Entity Type	Partition Key (PK)	Sort Key (SK)	GSI1 Partition (GSI1PK)	GSI1 Sort Key (GSI1SK)	GSI2 Partition (GSI2PK)	GSI2 Sort Key (GSI2SK)
User Metadata	USER#<user_id>	METADATA	ROLE#<role>	USER#<user_id>	COGNITO#<sub>	—
Worker Profile	WORKER#<worker_id>	PROFILE	SERVICE#<service_type>	RATING#<rating>	AREA#<area>	—
Booking Item	BOOKING#<booking_id>	METADATA	STATUS#<status>	DATE#<date>	CUSTOMER#<id>	—
Booking Note	BOOKING#<booking_id>	NOTE#<note_id>	—	—	—	—
Review / Rating	WORKER#<worker_id>	REVIEW#<review_id>	—	—	CUSTOMER#<id>	—
Activity Log	—	—	—	—	—	—

3.2.4 Deployment and Scaling

Connect360 is a serverless AWS application, which is deployed using Terraform as Infrastructure-as-Code. The Amazon API Gateway exposes the REST API endpoints over HTTPS and routes the incoming requests to the backend AWS Lambda functions. Frontend is hosted on Amazon S3, which is distributed over the internet using Amazon CloudFront over HTTPS.

The Amazon DynamoDB employs the On-Demand mode to fulfill the varying needs of the application without manual provisioning of database capacity. Being serverless, the Connect360 application can support varying traffic demands through the scaling capabilities of AWS Lambda. The Connect360 service employs different AWS Lambda functions with distinct computation

capacities for the authentication, Worker management, booking, verification, administration, and AI-Assistant processes.

The deployment architecture also uses AWS Secrets Manager to store secrets such as the API Gateway credentials. In addition, the external platforms including OSRM, Nominatim, and Google Gemini offer routing, geocoding, and conversational artificial intelligence capabilities, respectively. Overall, the deployment of Connect360 application utilizes Infrastructure-as-Code in conjunction with managed AWS services to offer a controlled environment for deploying the application.

3.3 SYSTEM DESIGN DIAGRAMS

3.3.1 Use Case Diagram

Figure 3.2 below represents the detailed use case diagram for the Connect360 system, including the interaction between the Customer, Skilled Worker, and Platform Admin, as well as the main functions provided by the platform.

Therefore, the Customer can register and authenticate, consult the AI assistant for service triage, browse/search workers, request Priority Booking, track the service route and ETA with polling-based mechanism, submit reviews and ratings, and direct customer-worker calling for active bookings. The Skilled Worker can edit their profile and skills, process booking requests, atomic acceptance of Priority Booking jobs, complete jobs and participate in authorized customer-worker communication. In addition, the Platform Admin is responsible for worker verification and skill-document review, as well as the administrative management of the platform.

The figure also shows important relationships within the system. For example, Consult AI Assistant (Triage) includes Deterministic PII Defense, while Browse & Search Workers includes Score & Rank Candidates. Moreover, Request Priority Booking is associated with Atomic Accept Priority Job

workflow via an extend relationship. All of these relationships help indicate the required processing for conversational assistance, candidate ranking, and Priority Booking. Thus, the use of <include> and <extend> relationships adheres to standard UML use-case relationships.

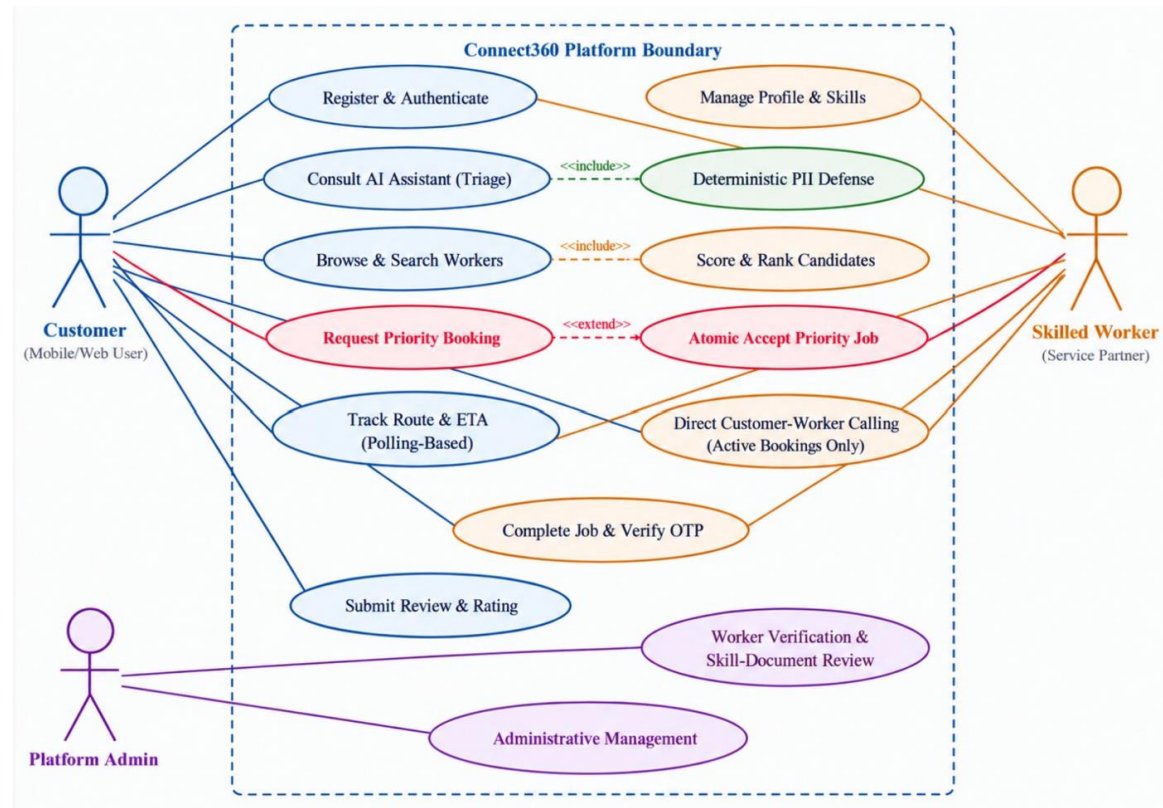


Figure 3.2: Connect360 Use Case Diagram

3.3.2 Activity Diagram

Figure 3.3 illustrates the detailed priority booking activity flow using the swim lanes for the Customer, Dispatch Service, Amazon DynamoDB, and Workers Pool. It starts with the customer selecting the emergency service and entering the GPS location. The dispatch service then performs the candidate ranking by the booking priority and stores the booking in DynamoDB with a worker_pending state.

The Dispatch service propagates the job alert to the eligible workers who check the job and location before requesting to accept the job. The system runs

a conditional write with an `attribute_not_exists(worker)` as a condition. An atomic lock decides whether to accept the worker assignment to a job or not. If the request meets the condition, it updates the booking to an accepted status, solves the OSRM road polyline, sends the worker-assigned alert to the customer with the booking status and the route. The system then proceeds to the job in progress step.

In a scenario where more than one worker requests the job at the same time, only one request will meet the condition and gain access to the atomic lock. The other request will encounter a failed condition and get aborted with an HTTP 409 conflict response. Thereafter, the system re-offers the job according to the above-mentioned workflow. DynamoDB conditional writes enable the system to perform a write operation if a specific condition is met.

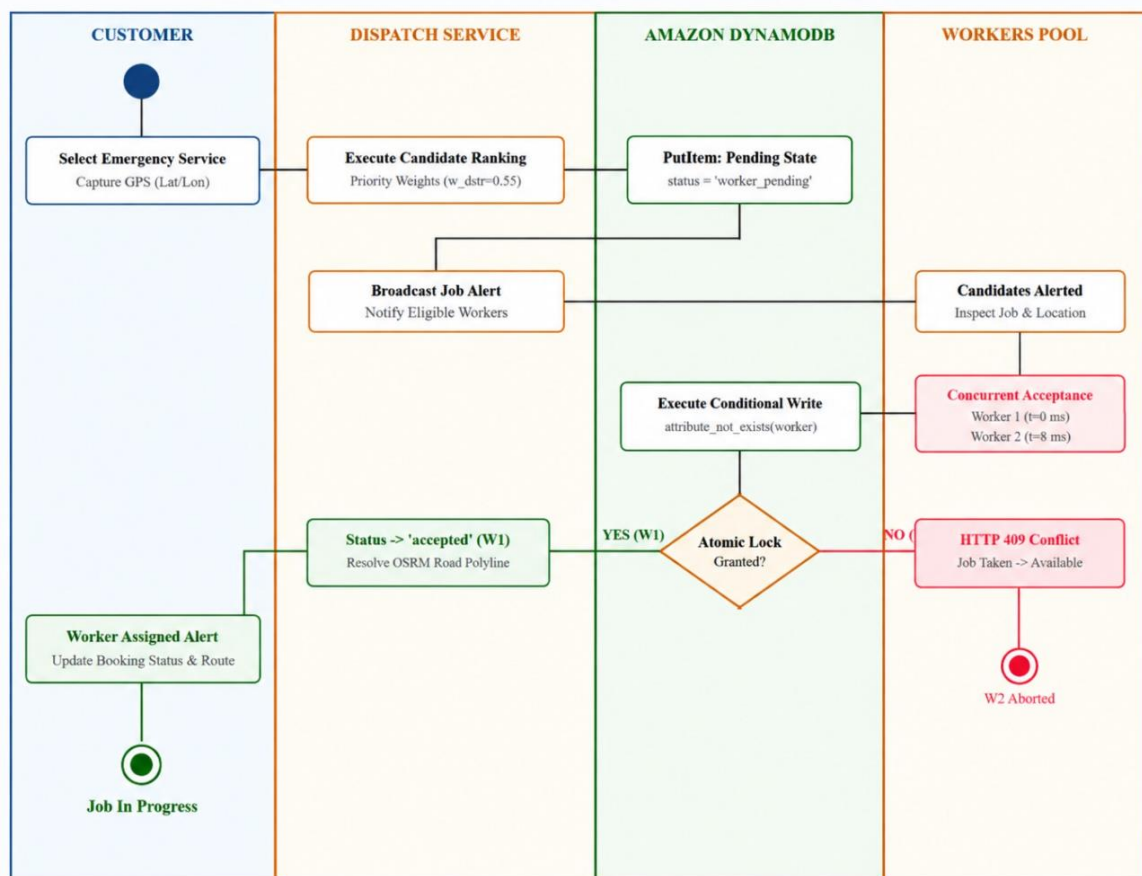


Figure 3.3: Connect360 Priority Booking Activity Diagram

3.3.3 Data Flow Diagrams

Figure 3.4 below represents the Level 0 Context Data Flow Diagram, which depicts the primary data flows between the Customer, Skilled Worker, Connect360 Marketplace Core entity, OSRM routing service, and AI Provider external services. The communications between Customers and Workers utilize credentials, worker profile and skill data, booking information, alerts for emergencies, location details, and service progress, while external services deliver road routing and conversational AI support.

The Level 1 Diagram is demonstrated in Figure 3.5 below, which decodes the modular decomposition of the Connect360 system into six major processing modules, including Authentication & RBAC Control, Worker Profile & Geospatial Index, Candidate Scoring, Priority Dispatch & Atomic Locking, Spatial Routing & Polling-Based Tracking, and AI Triage & PII Defense Filter. The Candidate Scoring module performs an $O(N)$ candidate-scoring stage, which passes the ranked list to the Priority Dispatch module, while the dispatch module utilizes DynamoDB conditional operations for atomic worker acceptance.

The figure also depicts the D1: DynamoDB Single-Table data store that stores users, workers, bookings, GSI1, and GSI2, and D2: Road Polyline Cache for the in-memory OSRM route information. The OSRM Engine provides road geometry for the Spatial Routing module, whereas the AI Provider delivers conversational triage through the AI Triage and PII Defense Filter. The data flows between modules represent the authentication information, worker and location data, ranked candidates, conditional locks, coordinates, sanitized prompts, and route information processed within the Connect360 system.

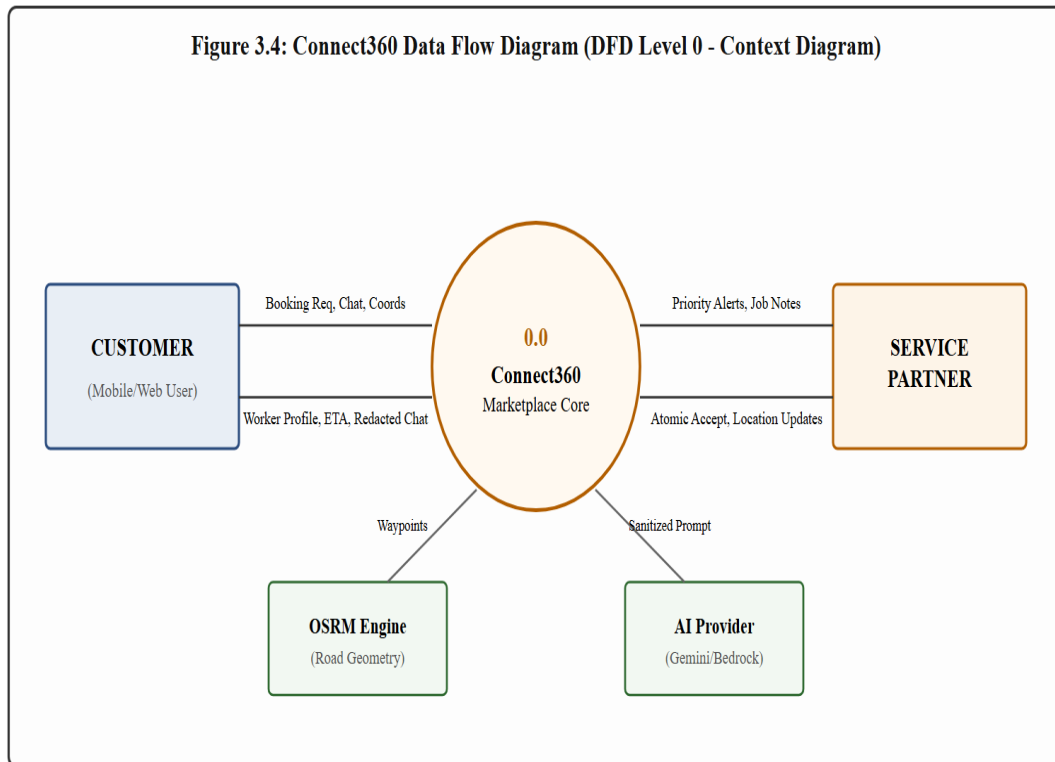


Figure 3.4: Connect360 Data Flow Diagram (DFD Level 0 - Context Diagram)

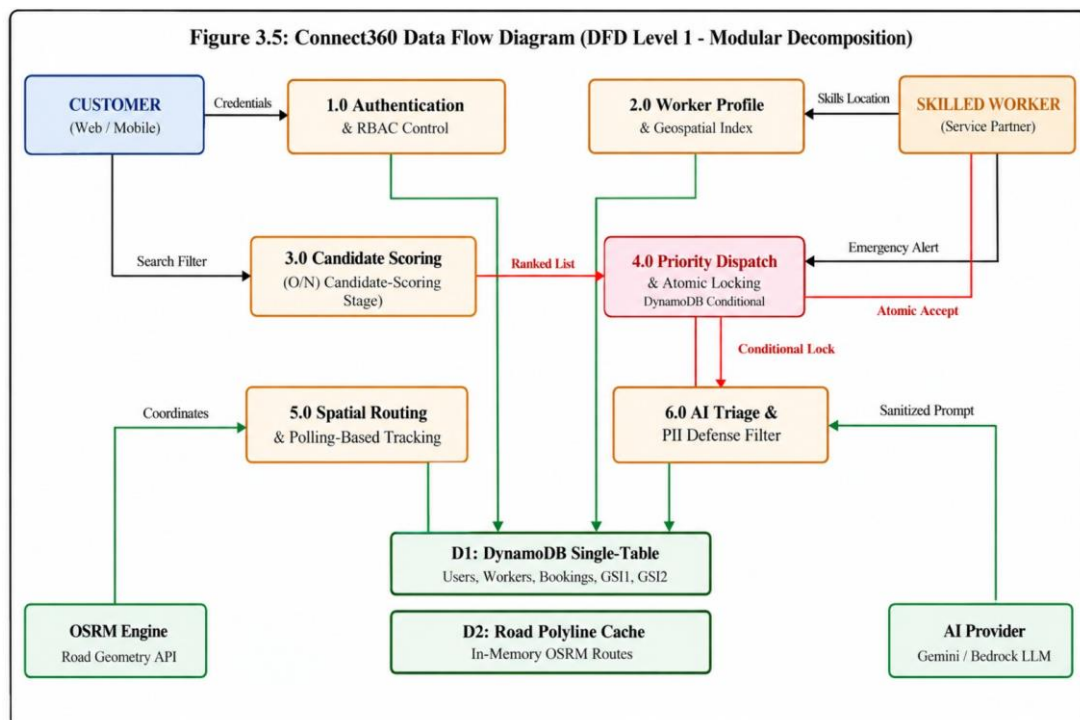


Figure 3.5: Connect360 Data Flow Diagram (DFD Level 1 - Modular Decomposition)

3.3.4 Sequence Diagram

The figure below represents the flow of events for a concurrent acceptance of a Priority Booking. The Customer requests a booking in the Client Application that is sent through the API Gateway into the Dispatch Service. The Dispatch Service writes the booking into the DynamoDB with status open under worker_pending and notifies the eligible workers about the job opening. While the workers are trying to concurrently accept the job, the DynamoDB updates the booking with a conditional update to set the status to accepted and assign the worker who sends the first successful accepted request.

The other concurrent requests of Accept Booking will result in a ConditionalCheckFailedException because of the failed conditional update and will respond to the client with HTTP 409 Conflict that the job is already taken. After the successful assignment of the worker to the booking, the client will receive the Worker Assigned Event and poll until it receives an updated booking, route, and ETA status.

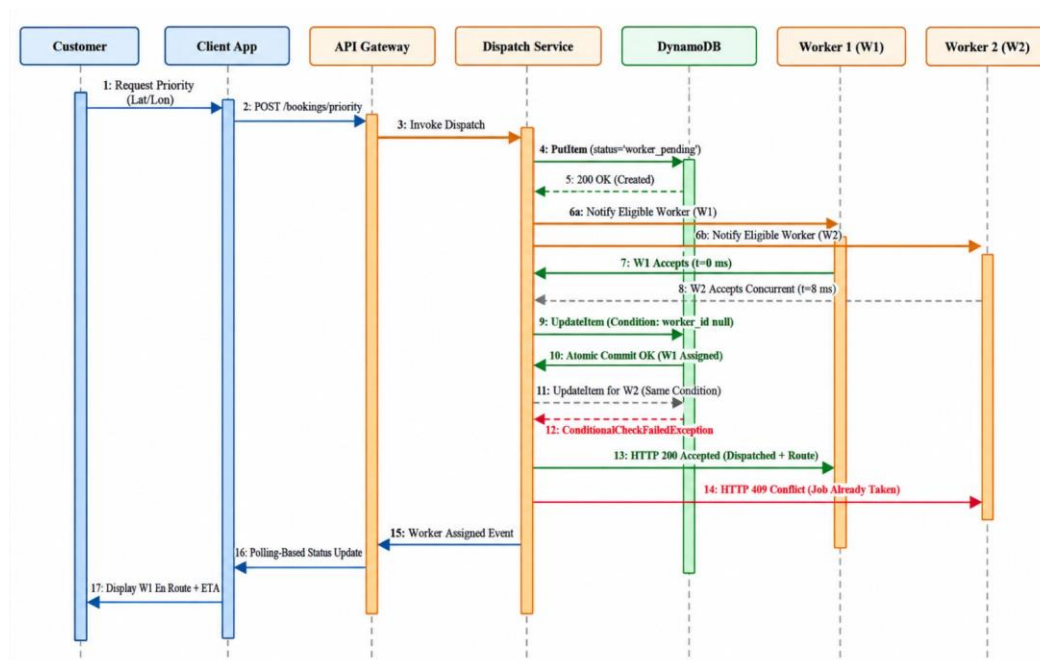


Figure 3.6: Sequence Diagram: Priority Booking Atomic Acceptance & Commit

CHAPTER 4

PROJECT DESCRIPTION

4.1 METHODOLOGIES

Connect360 was developed utilizing a cloud-based serverless development model, with modular backend services and independently deployable application functions. The following design principles were applied to the developed system:

Stateless Compute Architecture: The serverless functions (Lambda) are stateless, with the authentication and role information encoded in the HTTP request header in the form of Amazon Cognito signed JSON Web Tokens. When processing an incoming request, the backend Lambda function validates the JWT signature and extracts the identity and roles information from the JWT claims. The information is then used to authorize and process the request. This design enables each serverless function to handle HTTP requests independently.

Single-Table NoSQL Design: Application data is stored in Amazon DynamoDB using a single-table design with Global Secondary Indexes to manage user accounts, workers, bookings, reviews, etc. The structure supports the required query patterns for managing the worker directory and processing bookings.

Mathematical Multi-Criteria Weight Reallocation for Ranking: Worker directory search results are ranked using a mathematical formula that evaluates six (6) factors on a normalized scale, which are then weighted to produce an aggregate score. These factors include service match, trust, experience, availability, reliability, and distance. The relative weights are adjusted depending on the type of booking (Normal or Priority). For Normal

Booking, the service suitability and trust scores are prioritized, whereas for Priority Booking, availability and distance scores receive higher weights.

Deterministic AI Response Filtering: Connect360 utilizes a conversational assistant to facilitate service interaction and booking support. Google Gemini processes the information provided by the users and extracts booking-related details to assist in the booking process. As language-model responses are non-deterministic, deterministic filtering rules are applied to the response before it is relayed back to the users. The filtering engine identifies contact-related information such as phone numbers and email addresses and replaces them with [hidden] to prevent accidental exposure of contact details.

4.2 MODULES

4.2.1 Module 1: Authentication and Role-Based Access Control

This application component is responsible for handling user account registration, authentication, and identity management. It makes use of Amazon Cognito User Pools to host user accounts. Upon a successful authentication, the user receives signed JSON Web Tokens (JWT) with information about the user's identity and roles. The backend Lambda functions then verify the JWT signature and extract the user's unique account ID and roles information from the JWT claims.

The roles-based access control (RBAC) mechanism is then applied to manage access to the application components. The roles include Customer, Worker, Administrator, etc., with access to different system functions managed depending on the user's role. For example, a customer would have access to the booking-related features whereas the workers would have access to the worker profile management and verification-related components.

4.2.2 Module 2: Worker Profile & Geospatial Registration

This application component manages worker-related information such as worker profile, skills, experience, availability, service, and verification information. It makes use of Amazon S3 to store worker verification documents and Amazon DynamoDB to store worker profile information.

When registering or updating the worker location information, the worker directory component utilizes the Nominatim geolocation API to convert the human-readable address to geographic coordinates. The latitude and longitude information are then stored as part of the worker's profile information in DynamoDB. The availability, service, worker experience, ratings, and other relevant information are similarly stored and linked to the worker's profile using the worker's unique identifier. The information is then used to power the candidate discovery and matching algorithms.

The worker directory further allows filtering the candidate list by service, experience, availability, and other related factors based on the worker's profile. This information is used by the Candidate Matching Engine to rank the most suitable candidates for a given service request based on a combination of factors including service match, trust, experience, availability, reliability, and distance.

4.2.3 Module 3: Intelligent Matching Engine

The candidate discovery and ranking component utilizes a combination of factors to rank potential candidates for a given service request. This includes the following:

Service Match Score (M): This measures the extent to which a candidate's service category and skill matches the requested service type.

Trust Score (T): This represents the candidate's trustworthiness and is determined by their normalized rating and the review verification factor.

Experience Score (E): This indicates the candidate's level of relevant experience.

Availability Score (A): This is a binary metric that indicates whether the candidate is available for the requested job.

Reliability Score (R): This shows the worker's reliability based on their historical job completion performance.

Distance Score (D): This is determined by the proximity of the candidate to the customer's location. Workers who are closer receive a higher score.

With this information, an aggregate score is computed for all eligible candidates using the following formula:

$$S = w_1M + w_2T + w_3E + w_4A + w_5R + w_6D$$

Where:

M = Service Match Score

T = Trust Score

E = Experience Score

A = Availability Score

R = Reliability Score

D = Distance Score

$w_1 \dots w_6$ = Weights

The weight factors for each component are adjusted depending on the type of booking. For instance, for Normal Booking, higher weights are assigned to the service match and trust components whereas lower weights are assigned to the distance and availability. On the other hand, for Priority Booking, higher weights are assigned to the availability and distance components.

With the scores computed for each candidate, the candidate matching engine then ranks the candidates in descending order of their aggregate scores. This ranked list is then used to determine which workers should be invited to perform the job.

Table 4.1: Multi-Criteria Weighting Configurations for Normal vs. Priority Booking

Weight Parameter	Normal Booking Weight (w)	Priority Booking Weight (w)	Operational Rationale
Service Match Weight (w_m)	0.20	0.10	Ensures the worker's skills and service category match the requested service.
Trust Weight (w_t)	0.30	0.15	Considers worker reputation and verification status for reliable service quality.
Experience Weight (w_e)	0.15	0.05	Reflects relevant years of experience in the service domain.
Availability Weight (w_a)	0.10	0.30	Prioritizes workers who are currently available to accept the request, especially for urgent bookings.
Reliability Weight (w_r)	0.10	0.05	Rewards workers with higher job completion rates and lower cancellation history.
Distance Weight (w_d)	0.05	0.25	Gives greater importance to geographical proximity for faster response in Priority Booking.

4.2.4 Module 4: Transactional Priority Dispatch

The Priority Dispatch Handler manages the lifetime of Priority Bookings. When a customer submits a priority request, the booking is created with the appropriate pending state according to the configuration, and potential workers are found using the Priority Booking matching strategy. It is then offered to the matched worker.

Server-side availability checks are made when trying to accept a request, to ensure that it is still available and in the correct state.

The actual booking plus worker state transitions happen using the described DynamoDB writes. When contention happens due to simultaneous accepts, the first one gets a successful conditional update, while the second throws an error, which is then handled by returning an HTTP 409 Conflict to the client, noting that the job was already accepted. The Priority Dispatch Handler thus

makes use of server-side conditional and transactional operations for concurrency control.

4.2.5 Module 5: Hybrid Spatial Routing & Polling-Based Tracking

This module is responsible for candidate ranking based on simple spatial distance calculations while also making use of routing services for route calculation and polling-based tracking.

The Candidate Matching Engine makes use of simple spatial calculations to efficiently rank candidates based on their geographic distance to the customer. Once a candidate is chosen for a booking, the Spatial Routing Service can be utilized to receive road-based routing geometry, distance, and additional details.

Nominatim is used for address-to-coordinate lookups while Leaflet.js is used as the mapping library to display routing information to the customer. The client is responsible for periodically querying for booking and location updates while also refreshing the display with updated routing information. If the workers position differs significantly from the previously known position, routing information can be refreshed to update the geometry and drive time.

4.2.6 Module 6: Conversational AI Assistant & Contact-Information Redaction

The Conversational AI Assistant Module is used to power the various conversational interfaces used throughout the application. Gemini is used to interpret user messages and extract the relevant information such as booking type, urgency, location, date, time, and confirmation. Conversation history and related context are kept for the conversation to maintain statefulness. The assistant is capable of processing image input when needed, and a rules-based fallback is used in case the external AI service is unavailable.

The assistant's responses undergo deterministic post-generation filtering before being sent to the client. This acts as an extra layer of security outside the control of the language model to prevent unwanted information such as contact details to be sent. This filtering process is completely separate from the language model itself but is applied to its responses. Any phone numbers and email addresses detected get replaced with the [hidden] placeholder. The post-generation filtering acts as an additional layer of protection for the application's users and is particularly useful when unwanted contact information gets accidentally exposed in the model's response.

4.2.7 Module 7: Controlled Customer-Worker Calling

This module delivers the calling feature for cases when the customer wishes to contact the worker directly. The CallButton to initiate a call is only displayed if a booking is active, and the logged-in user is an authorized participant of the booking. When the authorized participant selects the call option, the application uses the device's default dialer to initiate a call.

This implementation does not utilize any third-party voice calling services such as Twilio Voice or Cloud PBX solutions. The backend enforces role-based access and booking ownership to ensure that only the customer who requested the service and the worker assigned to the booking can access the calling functionality. The contact information is not displayed through worker cards, listings, or other public-facing elements. Calling access is therefore restricted to the active booking context, providing controlled customer-worker communication during the service.

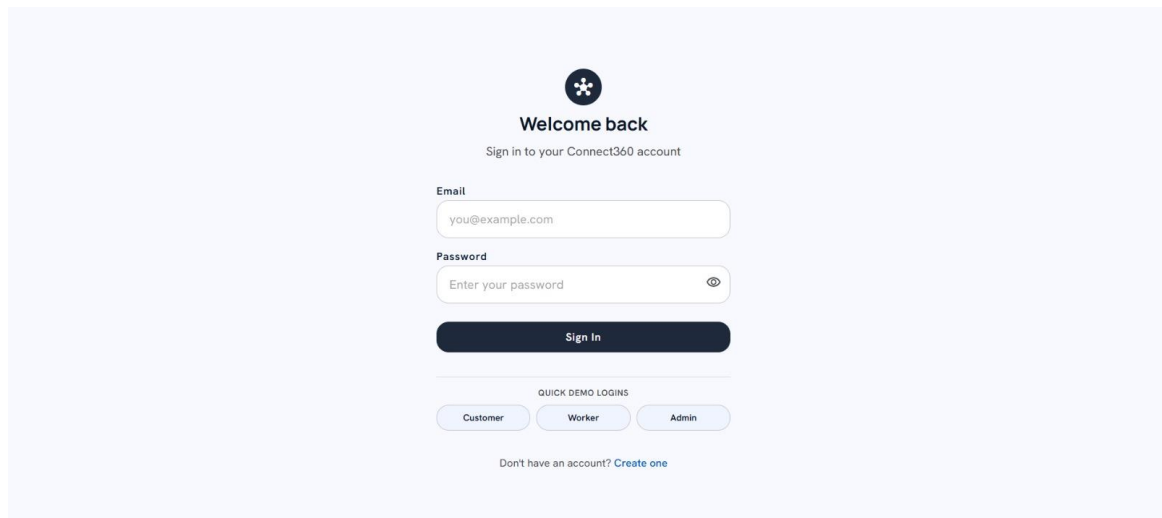
CHAPTER 5

IMPLEMENTATION AND RESULT DISCUSSION

5.1 IMPLEMENTATION RESULTS

The Connect360 platform was implemented as a serverless cloud application using React as the front-end and AWS back-end with external routing and AI components. The front-end offers role-specific interfaces to customers, workers, and administrators. It includes worker discovery, booking management, Priority Booking, AI-assisted conversation, and polling-based route tracking features. The back-end is formed with domain-specific AWS Lambda functions for authentication, worker management, booking, verification, administration, and conversational assistance.

The implemented application was evaluated using controlled test cases and synthetic datasets to investigate the main functional and performance characteristics of the proposed system. Experiments were performed in a controlled development/test environment as compared to measuring production-ready deployments. Synthetic worker records, booking requests, geographical coordinates, concurrency requests, and AI test cases were generated according to specific experiments' requirements. The evaluation focused on performance comparisons, ranking sensitivity, concurrency safety, spatial distance comparison, AI response safety, structured-output conformity, and overall system reliability.




Welcome back
 Sign in to your Connect360 account

Email

Password

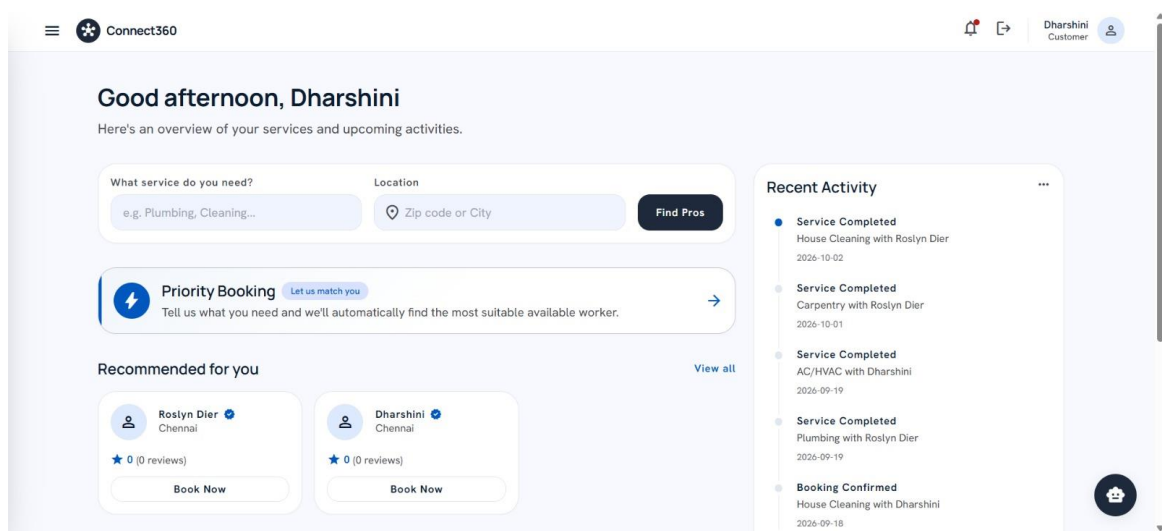
Sign In

QUICK DEMO LOGINS

[Customer](#)
[Worker](#)
[Admin](#)

Don't have an account? [Create one](#)

Figure 5.1: User Login Interface



Connect360 Dharshini Customer

Good afternoon, Dharshini
 Here's an overview of your services and upcoming activities.

What service do you need?
e.g. Plumbing, Cleaning...

Location
Zip code or City

Find Pros

Priority Booking Let us match you
 Tell us what you need and we'll automatically find the most suitable available worker.

Recommended for you [View all](#)

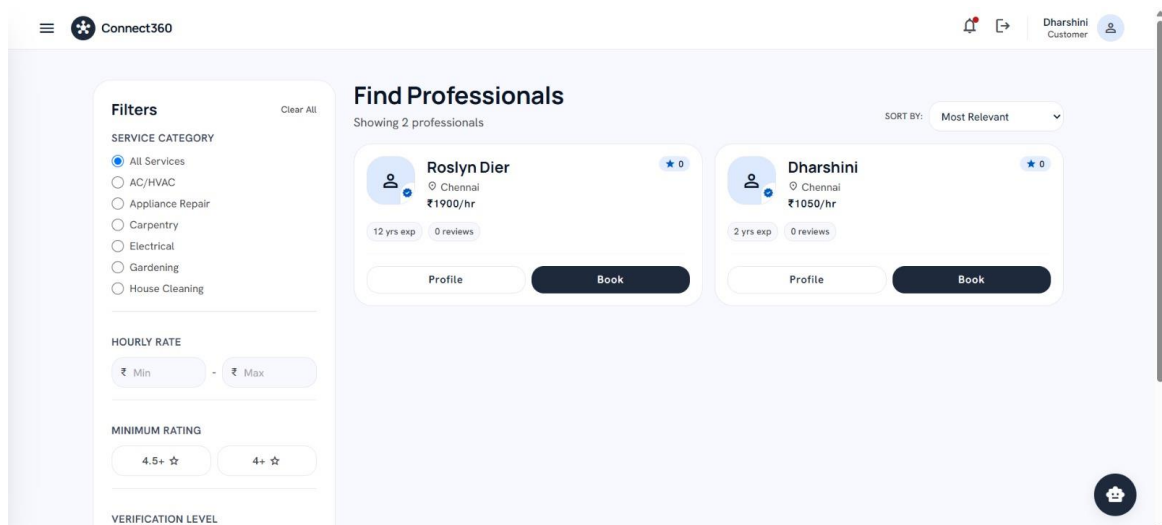
Roslyn Dier
Chennai
★ 0 (0 reviews)
Book Now

Dharshini
Chennai
★ 0 (0 reviews)
Book Now

Recent Activity

- Service Completed**
House Cleaning with Roslyn Dier
2026-10-02
- Service Completed**
Carpentry with Roslyn Dier
2026-10-01
- Service Completed**
AC/HVAC with Dharshini
2026-09-19
- Service Completed**
Plumbing with Roslyn Dier
2026-09-19
- Booking Confirmed**
House Cleaning with Dharshini
2026-09-18

Figure 5.2: Customer Dashboard



Connect360 Dharshini Customer

Filters [Clear All](#)

SERVICE CATEGORY

- ☒ All Services
- ☐ AC/HVAC
- ☐ Appliance Repair
- ☐ Carpentry
- ☐ Electrical
- ☐ Gardening
- ☐ House Cleaning

HOURLY RATE

₹ Min - ₹ Max

MINIMUM RATING

4.5+ ★ 4+ ★

VERIFICATION LEVEL

Find Professionals
 Showing 2 professionals

SORT BY: Most Relevant

Roslyn Dier
Chennai
₹1900/hr
12 yrs exp 0 reviews
Profile **Book**

Dharshini
Chennai
₹1050/hr
2 yrs exp 0 reviews
Profile **Book**

Figure 5.3: Find Professionals Interface

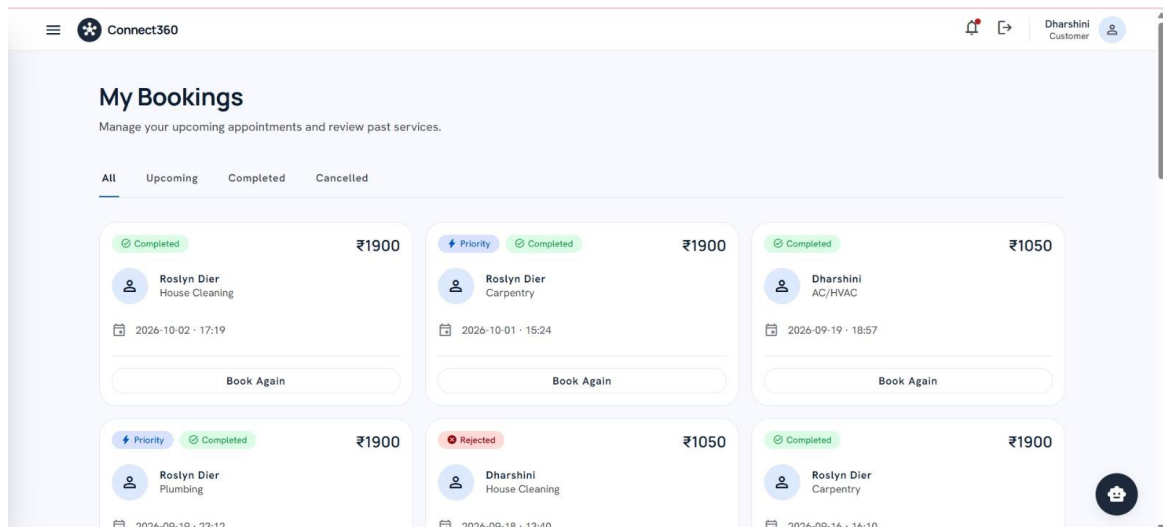


Figure 5.4: Customer Booking History

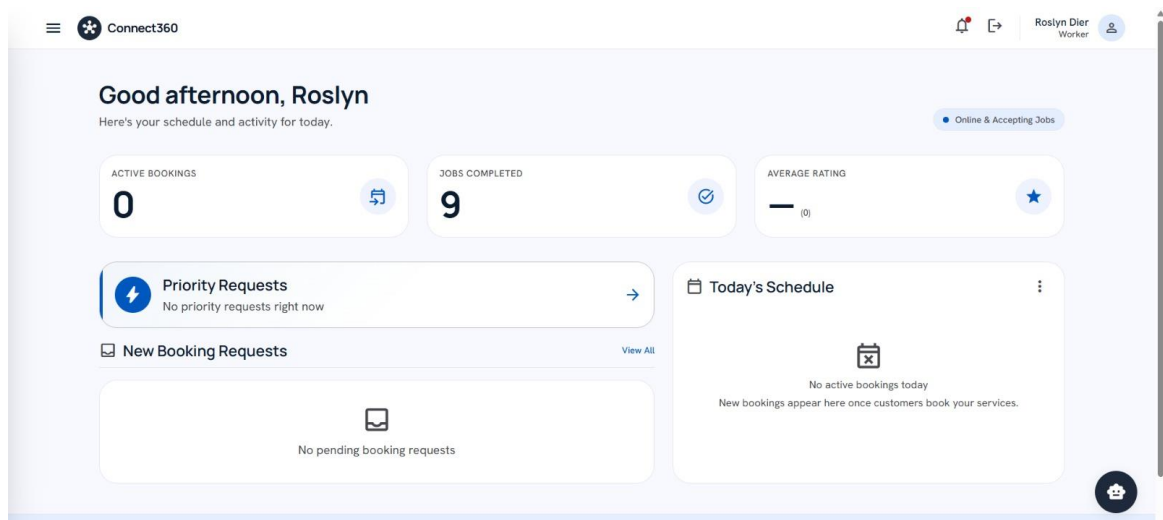


Figure 5.5: Priority Booking Request

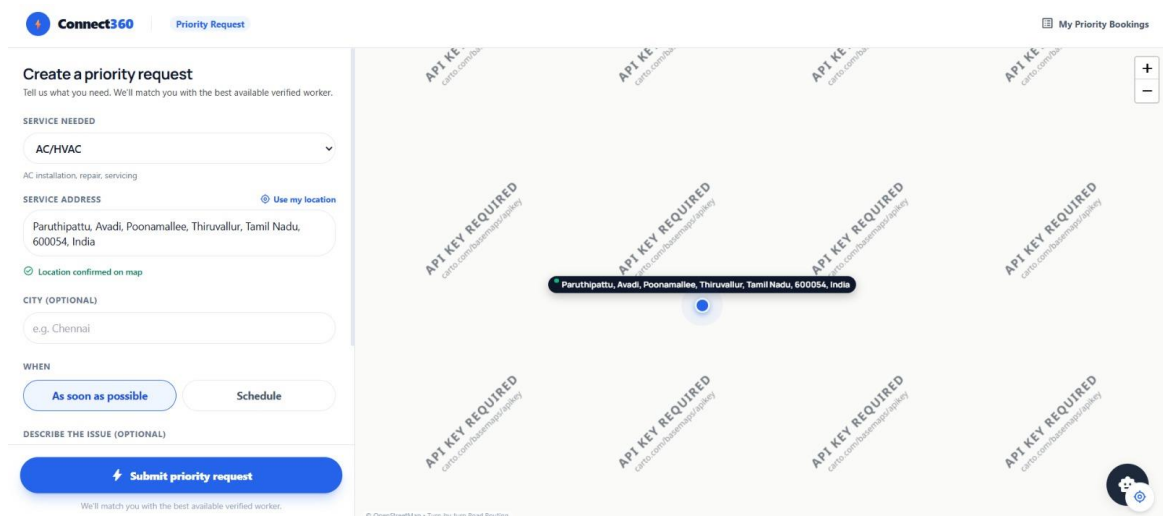


Figure 5.6: Customer Priority Booking Tracking

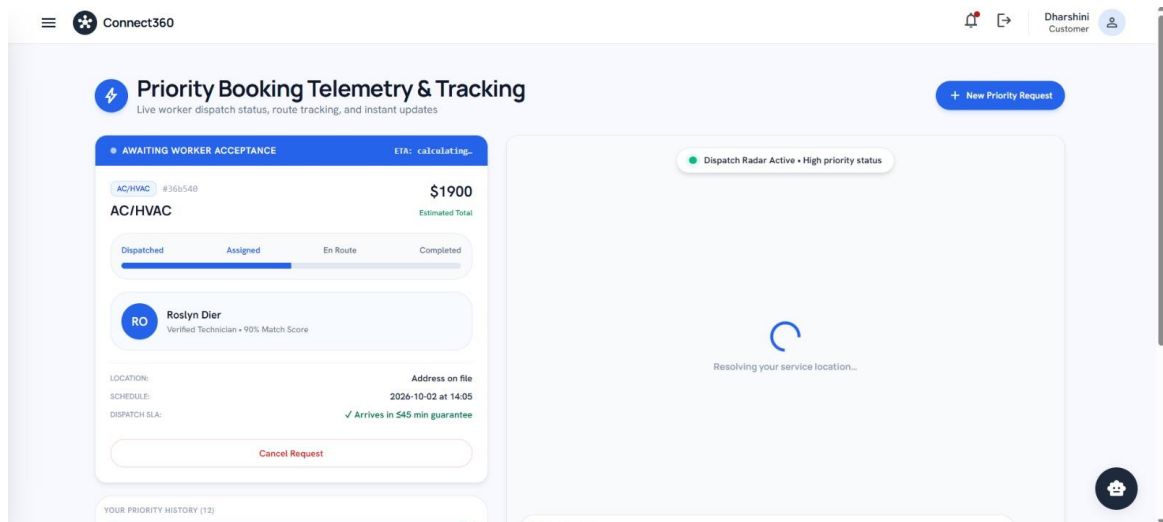


Figure 5.7: Worker Dashboard

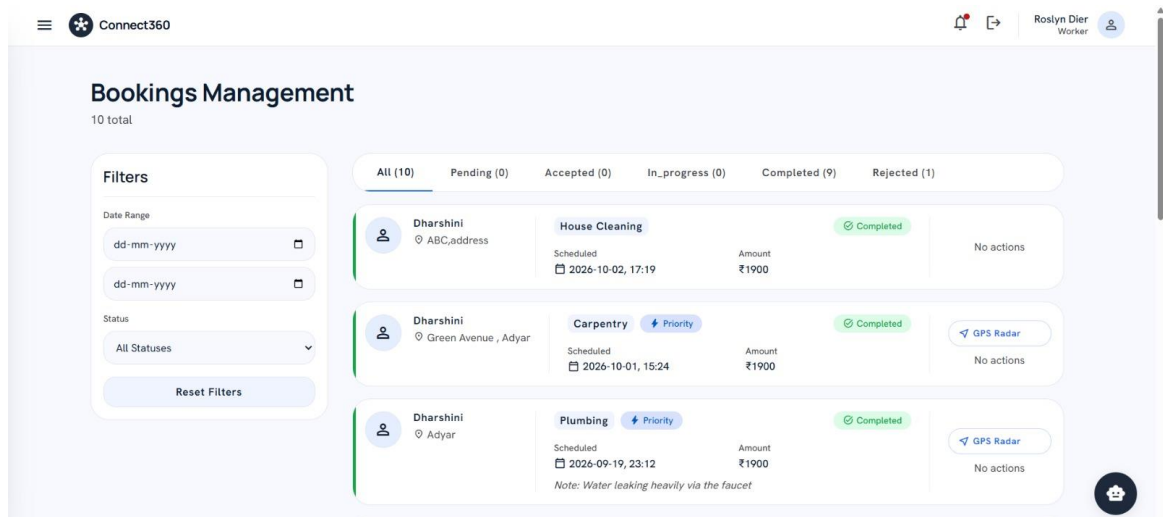


Figure 5.8: Worker Bookings Management

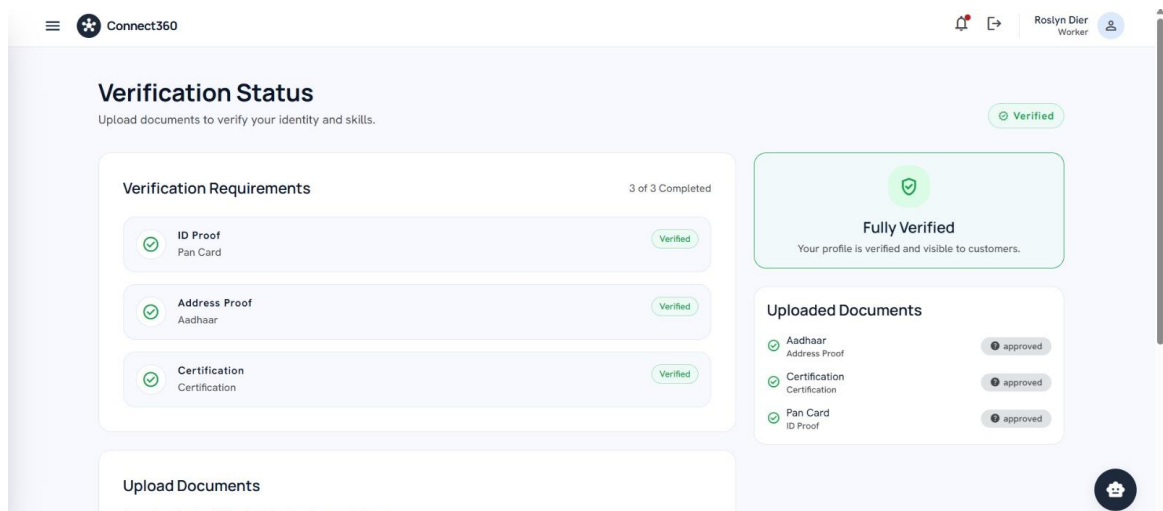


Figure 5.9: Worker Verification Status

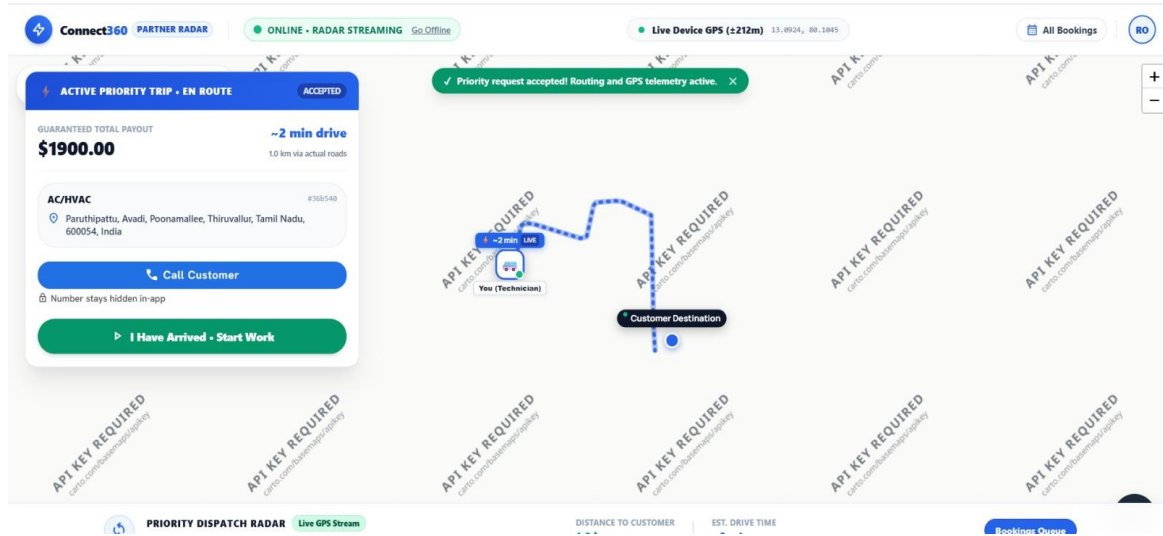


Figure 5.10: Priority Dispatch Radar and Polling-Based Tracking

5.2 EVALUATION AND PERFORMANCE ANALYSIS

5.2.1 Metric 1: Matching Algorithm Execution Latency vs. Candidate Pool Size

Evaluation Objective

The purpose of this experiment is to evaluate the execution latency and scaling performance of the Connect360 multi-criteria Candidate Matching Engine for different possible candidate pool sizes under Normal and Priority booking configurations. In other words, this experiment attempts to quantify how ranking scales with the size of the candidate pool.

Experimental Setup

Several candidate pools of sizes $N = \{10, 50, 100, 250, 500, 1000, 2500, 5000\}$ were created by randomly selecting worker records with randomized attributes and coordinates over a fictional city. Every candidate was given attributes, a rating, experience, availability, reliability, and location according to the six-component scoring model described in this paper. For each candidate pool size, one thousand algorithm runs were made under both Normal and Priority

configurations, and the execution time was measured using the high-resolution `time.perf_counter_ns()` function to define the ranking latency.

As can be seen in the plots above, this experiment measured the computational performance of the matching engine at different candidate pool sizes. The candidate-scoring stage has linear complexity since every candidate is scored once for every criterion. The whole score-and-sort pipeline, however, has $O(N \log N)$ complexity because of the sorting stage that follows the scoring stage to rank all candidates according to their scores.

Table 5.1: Matching Algorithm Execution Latency Across Candidate Pool Sizes

Candidate Pool Size (N)	Normal Mean (ms)	Normal P90 (ms)	Normal P99 (ms)	Priority Mean (ms)	Priority P90 (ms)	Priority P99 (ms)
10	0.007	0.010	0.019	0.008	0.011	0.021
50	0.035	0.046	0.076	0.038	0.050	0.082
100	0.071	0.093	0.147	0.076	0.099	0.160
250	0.178	0.228	0.354	0.191	0.244	0.378
500	0.360	0.457	0.686	0.384	0.485	0.741
1000	0.728	0.916	1.344	0.772	0.969	1.442
2500	1.834	2.296	3.284	1.942	2.417	3.502
5000	3.682	4.588	6.419	3.894	4.829	6.786

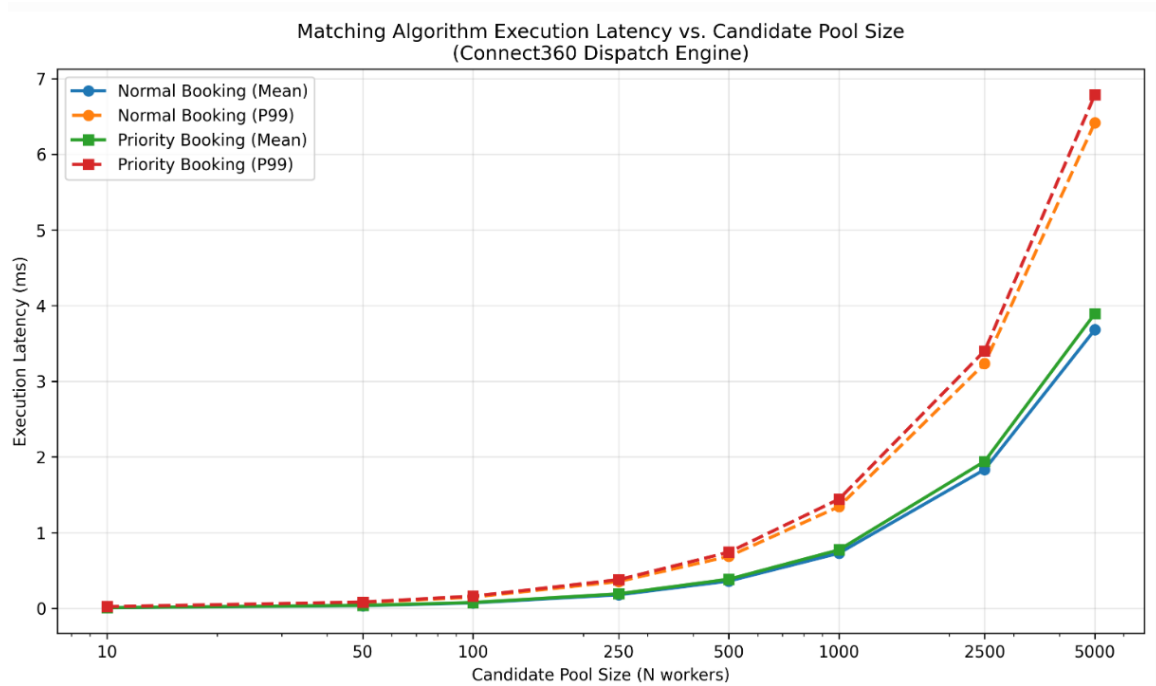


Figure 5.11: Matching Algorithm Execution Latency vs. Candidate Pool Size

Discussion and Trends

As shown in Figure 5.11, the measured execution latency increases approximately with the candidate pool size. The candidate-scoring stage remains linear in the number of candidates, while the complete ranking process also includes the ordering operation. For a candidate pool of $N=100$, the measured mean execution latency was 0.071 ms for Normal Booking and 0.076 ms for Priority Booking, as reported in Table 5.1.

Across this range, the latency grows approximately $0.73 \mu\text{s}$ per candidate for the Normal configuration. The increase is therefore broadly proportional to the number of candidates in the measured benchmark, without a sudden computational escalation at a particular pool size. At $N=5000$, the measured mean latency was 3.682 ms for Normal Booking and 3.894 ms for Priority Booking.

The Priority Booking configuration therefore has a small increase in the measured execution time compared with Normal Booking. Since both configurations evaluate the same six scoring components and differ mainly in their weighting coefficients, this difference is not due to a fundamentally different scoring procedure. The observed difference however could instead be measured variation and execution-level overhead in the benchmark environment.

Overall, these results therefore show approximately linear measured latency growth as the candidate pool increases, while the matching architecture performs the same candidate scoring and ranking process under separate Normal and Priority Booking weighting configurations. The scoring stage is $O(N)$, whereas a full ordering of all candidates contributes the $O(N \log N)$ component to the complete ranking pipeline.

Limitations

This benchmark mostly measures the CPU-bound execution time of the in-memory Candidate Matching Engine. It does not represent complete end-to-end booking latency, which may additionally include database access, AWS Lambda execution overhead, API Gateway processing, network latency, and client-side processing.

The benchmark therefore describes the computational performance of the ranking operation rather than the response time of a complete production booking request.

5.2.2 Metric 2: Matching Engine Rank Sensitivity and Inversion Analysis

Evaluation Objective

The objective of this experiment is to assess the sensitivity and rank displacement of the matching engine when switching between Normal Booking and Priority Booking weight configurations. The experiment

examines how changes in the relative importance of service match, trust, experience, availability, reliability, and distance affect the ordering of candidate workers.

Experimental Setup

A fixed candidate pool of $N = 100$ synthetic workers was evaluated using the same worker attributes and geographic information under both Normal Booking and Priority Booking configurations.

The Normal Booking configuration used the weighting vector:

$$W_N = (0.30, 0.25, 0.15, 0.15, 0.10, 0.05)$$

corresponding respectively to service match, trust, experience, availability, reliability, and distance.

The Priority Booking configuration used:

$$W_P = (0.20, 0.15, 0.05, 0.30, 0.05, 0.25)$$

Rank displacement was calculated as:

$$\Delta Rank = Rank_{Normal} - Rank_{Priority}$$

A positive value indicates that a worker moves upward in the Priority Booking ranking, while a negative value indicates movement downward. Spearman's rank correlation coefficient and Kendall's rank correlation coefficient were also used to measure the overall agreement between the two rankings.

Table 5.2: Rank Inversion & Sensitivity Analysis Under Weight Reallocation

Distance Tier	Sample Count	Promoted Workers ($\Delta\text{Rank} > 0$)	Demoted Workers ($\Delta\text{Rank} < 0$)	Mean Rank Shift (ΔRank)
Near (< 5 km)	30	20 (66.7%)	6 (20.0%)	+6.47
Mid (5–12 km)	40	18 (45.0%)	17 (42.5%)	+0.15
Far (12–25 km)	30	5 (16.7%)	21 (70.0%)	-6.67
OVERALL	100	43 (43.0%)	44 (44.0%)	Mean $\Delta\text{Rank} = 13.18$ (Max = 40) Spearman $\rho = 0.836$ Kendall $\tau = 0.650$

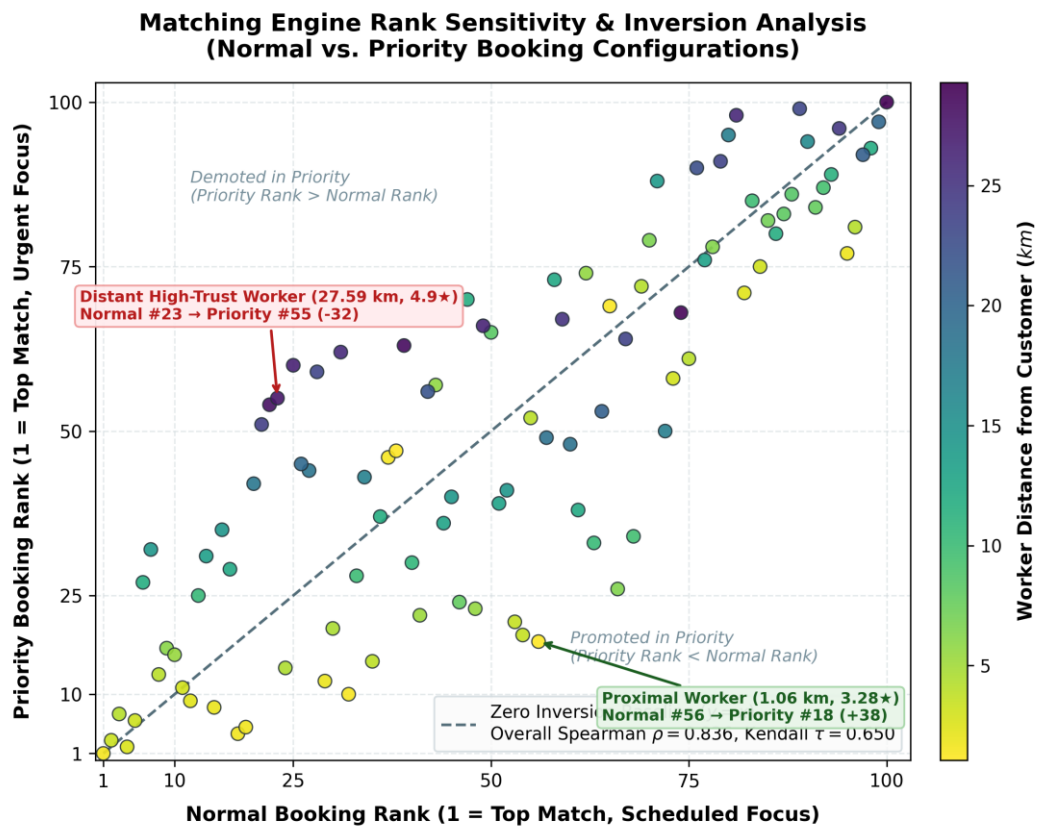


Figure 5.12: Matching Engine Rank Sensitivity & Inversion Analysis

Discussion and Trends

Figure 5.12 presents the change in candidate rankings resulting from the switch between the Normal and Priority Booking configurations. The

reallocation of the weights leads to a noticeable rank displacement since the availability and distance gain more relative importance under the Priority Booking.

The evaluation reported a mean absolute rank displacement of around 13.18 positions with the maximum observed displacement amounting to 40 positions. Therefore, the candidates with stronger proximity and availability characteristics can considerably climb up the Priority Booking ranking, while the candidates whose strengths are closely associated with general service quality and trust might substantially slip down.

The observed ranking changes indicate that the two booking configurations lead to different candidate orderings while retaining the same underlying multi-criteria ranking mechanism. This allows the matching engine to behave differently towards the normal service requests and the urgent requests without the need of a separate ranking architecture.

Limitations

The relative importance of the individual criteria is predetermined instead of being learned automatically from the historical booking data. As such, the weighting configuration represents a designed operational policy rather than a statistically learned model. Future work can investigate the adaptive weighting using larger historical datasets and additional contextual factors, such as traffic conditions, worker demand, response behaviour, and historical acceptance patterns.

5.2.3 Metric 3: Transactional Concurrency and Race Condition Prevention

Evaluation Objective

The objective of this experiment was to evaluate the ability of the Priority Dispatch Handler to maintain booking consistency under multiple concurrent

acceptance attempts. The evaluation focused on avoiding race conditions, assignment duplication, and double-booking of workers.

Experimental Setup

Concurrencies of

$C \in \{1, 2, 5, 10, 20, 50, 100\}$

were evaluated. Each concurrency level was tested across 50 trials, amounting to 9,400 concurrent booking requests in total.

The benchmark was performed in a controlled test environment using simulated concurrent requests that were not meant to be interpreted as a measure for production DynamoDB network latency. Multiple simulated workers attempted to accept the same pending Priority Booking concurrently, and the test recorded successful commits, conditional conflicts, and double-booking results.

An additional test case was included that had the same worker attempt to accept two different bookings concurrently. This test was intended to verify that worker availability conditions prevent the same worker from being assigned to multiple active bookings when the corresponding eligibility condition is no longer satisfied.

Table 5.3: Transactional Concurrency & Conflict Abort Latency Across Thread Loads

Concurrency (C)	Total Requests	Successful Commits	Conflict Aborts (409)	Double Bookings	Winning Latency (ms)	Abort Latency (ms)
1	50	50 (100.0%)	0 (0.0%)	0	0.212	N/A
2	100	50 (50.0%)	50 (50.0%)	0	0.235	0.024
5	250	50 (20.0%)	200 (80.0%)	0	0.264	0.028
10	500	50 (10.0%)	450 (90.0%)	0	0.281	0.032
20	1000	50 (5.0%)	950 (95.0%)	0	0.308	0.038
50	2500	50 (2.0%)	2450 (98.0%)	0	0.332	0.046
100	5000	50 (1.0%)	4950 (99.0%)	0	0.354	0.052
TOTAL	9400	350 (3.72%)	9050 (96.28%)	0 (0.00%)	Mean: 0.284 ms	Mean: 0.037 ms

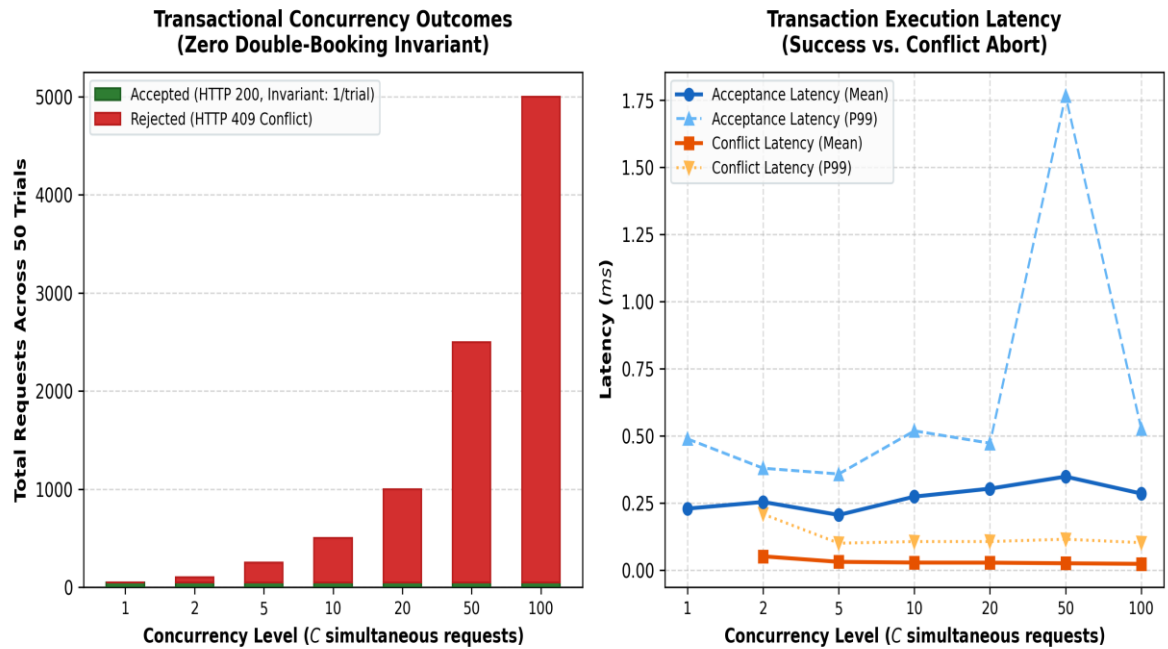


Figure 5.13: Transactional Concurrency & Race Condition Prevention

Discussion and Trends

As illustrated in Figure 5.13 , the evaluation yielded a 0.00% double-booking rate across the 9,400 concurrent simulated requests. At every contention level, only the successful acceptance of the appropriate booking and worker-state condition was allowed to commit, while the contending requests were rejected if the relevant condition was invalid.

The results are illustrative of the intended behaviour of the conditional and transactional mechanisms utilized in the Priority Dispatch Handler. While increasing the count of concurrent attempts increases the number of rejected conflicts, the tested booking state remains consistent and no duplicate booking assignments were observed.

The additional worker-contention test also validates the second aspect of the concurrency requirement: a worker that has already been assigned to an active booking should not be simultaneously committed to another booking when the worker eligibility condition prevents such an assignment.

Limitations

The aforementioned experiment was conducted in a controlled test environment and mainly evaluates logical booking consistency under the simulated concurrent requests. The measured execution latency should not be extrapolated into production DynamoDB network latency. Actual deployment performance may also vary according to AWS service-region selection, network conditions, Lambda execution overhead, API Gateway processing, and database request latency.

5.2.4 Metric 4: Road Distance vs. Haversine Disparity (Tortuosity Factor)

Evaluation Objective

The objective of this experiment is to quantify the disparity between straight-line Haversine distance and actual road-network distance across representative

metropolitan dispatch routes. The experiment evaluates the suitability of the hybrid spatial architecture in which lightweight geographic calculations are used during candidate matching and OSRM is used after worker assignment to obtain road-network routing information for navigation and polling-based route and status tracking.

The tortuosity factor is calculated as:

$$\tau = \frac{D_{OSRM}}{D_{Hav}}$$

where D_{OSRM} represents the actual road-network distance returned by the OSRM routing service and D_{Hav} represents the corresponding straight-line Haversine distance.

Experimental Setup

A total of 50 synthetically defined representative dispatch routes were evaluated using coordinate pairs representing locations across the Chennai metropolitan area. The routes were grouped into three distance tiers: inner-city routes below 6 km, mid-city routes from 6–12 km, and suburban routes from 12–20 km.

For each route, the Haversine distance between the origin and destination coordinates was calculated and compared with the corresponding road distance returned by the OSRM routing service. The resulting distance ratios were analysed to quantify how much straight-line distance differs from actual road-network distance.

Table 5.4: Haversine vs. OSRM Road Distance Disparity Across Distance Tiers

Distance Tier	Routes	Mean Haversine (km)	Mean OSRM Road (km)	Mean Disparity (km)	Mean Tortuosity (τ)	Score Distortion (ΔS)
Inner City (< 6 km)	14	3.62	4.94	+1.32	1.365	0.0528
Mid City (6-12 km)	21	8.85	11.38	+2.53	1.286	0.1012
Suburban (12-20 km)	15	15.24	19.12	+3.88	1.255	0.1552
OVERALL	50	9.30	11.90	+2.60	1.285 (≈ 1.30)	0.1020

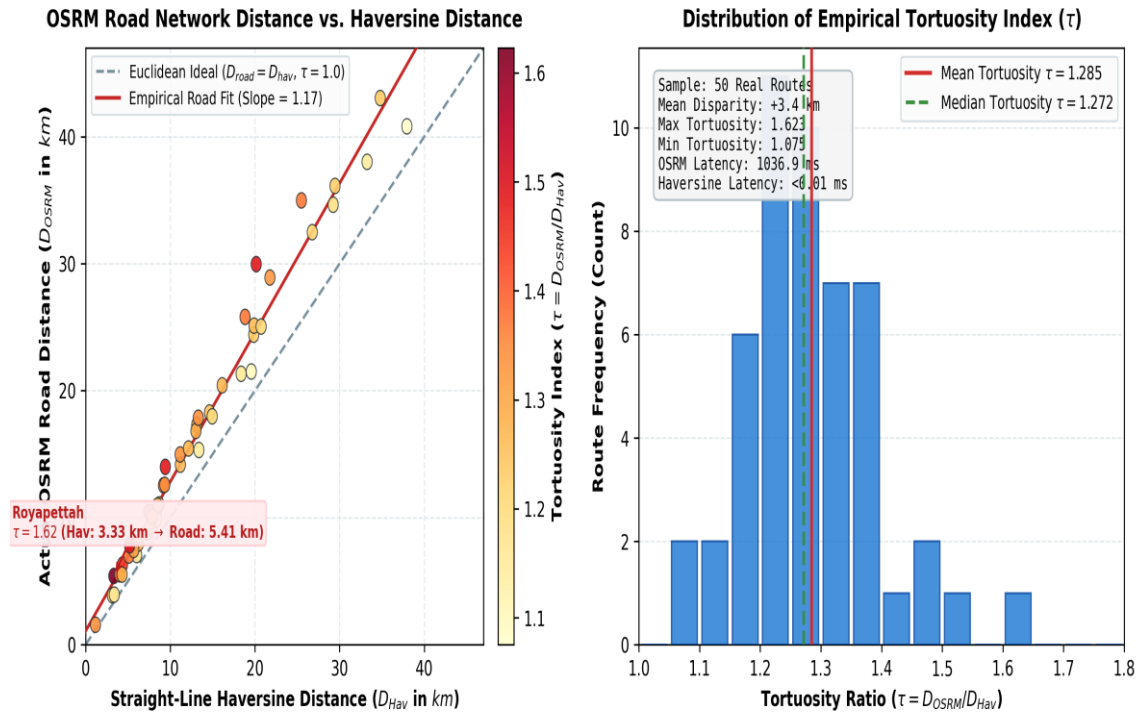


Figure 5.14: Road Distance vs. Haversine Disparity & Tortuosity Factor Distribution

Discussion and Trends

Figure 5.14 illustrates the relationship between Haversine distance and OSRM road distance. The weighted results from the three distance tiers produce an overall mean Haversine distance of approximately 9.30 km and an overall mean OSRM road distance of approximately 11.90 km.

The corresponding weighted mean disparity is approximately +2.60 km, indicating that the road-network distance was, on average, greater than the straight-line distance. The weighted mean tortuosity factor is approximately:

$$\tau = \frac{D_{OSRM}}{D_{Hav}} \approx 1.30$$

This corresponds to an average road distance approximately 28.5% greater than the corresponding Haversine distance for the evaluated sample.

The results demonstrate why straight-line distance is useful for computationally efficient candidate comparison but is not sufficient for representing actual travel distance. Road-network topology, connectivity, turns, barriers, and available routes can cause the actual travel path to differ substantially from the direct geographic distance.

Therefore, Connect360 uses a hybrid spatial approach. Haversine distance is used as a lightweight geographic criterion during candidate ranking, while OSRM is used after assignment to obtain road-network geometry and route information for navigation and polling-based route and status tracking.

The score distortion term ΔS represents the change in the worker's matching score that can result when the distance component is recalculated using road-network distance instead of straight-line distance. It can be expressed as:

$$\Delta S = S_{road} - S_{Hav}$$

where S_{road} is the score calculated using the road-distance value and S_{Hav} is the score calculated using the Haversine-distance value.

Limitations

The measured routing latency depends on the OSRM service configuration and network conditions during the experiment. Routing-server response time may vary according to server load, network conditions, geographic region, and request volume. Therefore, the observed routing latency should not be treated as a fixed value for every deployment environment.

5.2.5 Metric 5: Contact-Information Redaction Evaluation

Evaluation Objective

The objective of the experiment was to determine the precision, recall, specificity, F1-Score, and latency of the Connect360 deterministic contact-information redaction filter on a balanced corpus of $N = 500$ test cases containing sensitive contact information and normal operational text.

Testing Procedure

The test set used for the experiment contained 250 positive PII cases and 250 negative operational cases. The former contained Indian phone numbers, prefixed phone numbers, international format phone numbers, email addresses, and contact details embedded in a conversational script. The latter contained operational details such as prices, six-digit PIN codes, dates, technical specifications, and normal booking-related conversation script. The entire test set underwent multiple iterations of timing trials to determine detection performance and latency. True positive, true negative, false-positive, and false-negative rates were measured to determine the filter's recall, specificity, precision, and F1-score.

Table 5.5: Contact-Information Redaction Performance Across 10 Categories

Category	Class	Samples	TP	TN	FP	FN	Recall	Specificity	F1-Score	Mean Latency (µs)
Indian Phone Standard	PII	50	50	0	0	0	100.0%	—	1.0000	1.21
Indian Phone Prefix	PII	50	49	0	0	1	98.0%	—	0.9899	1.40
International Phone	PII	50	49	0	0	1	98.0%	—	0.9899	1.17
Email Address	PII	50	50	0	0	0	100.0%	—	1.0000	0.95
Dialogue PII Embedded	PII	50	50	0	0	0	100.0%	—	1.0000	2.67
Operational Prices	Benign	50	0	50	0	0	—	100.0%	—	2.40
Operational Pincodes	Benign	50	0	50	0	0	—	100.0%	—	1.94
Operational Specs	Benign	50	0	50	0	0	—	100.0%	—	2.42
Operational Dialogue	Benign	50	0	49	1	0	—	98.0%	0.0000	2.64
Operational Dates/Times	Benign	50	0	26	24	0	—	52.0%	0.0000	2.40
OVERALL	Mixed	500	248	225	25	2	99.20%	90.00%	0.9484	1.92

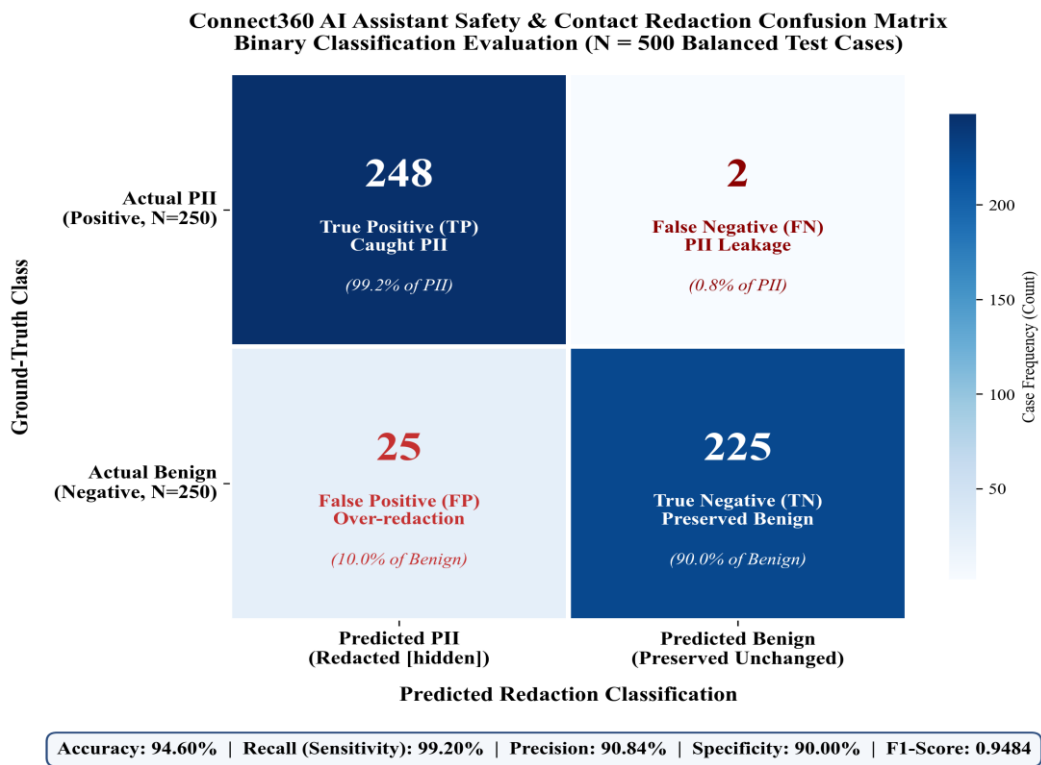


Figure 5.15: Connect360 Contact-Information Redaction Confusion Matrix

Discussion and Trends

As depicted in Figure 5.15, the deterministic contact-information redaction mechanism detected 248 out of 250 positive PII cases, thus providing an overall recall of 99.20%. The evaluation further included 225 true negatives and 25 false positives within the examined benign test cases.

An overall F1-score of 0.9484 was registered, whereas the measured mean filtering latency amounted to approximately 1.92 μ s per operation in the benchmark environment. The F1-score is consistent with the reported confusion-matrix counts of TP = 248, FP = 25 and FN = 2, per scikit-learn

The filter preserved several operational values without the need for redaction. Examples of such items were legitimate operational prices and six-digit PIN-code-like values, which were retained in the evaluated test cases. Yet, the evaluation further reported two false-negative cases associated with boundary-format phone patterns and 25 false-positives in general, including various overlaps with date-like or other hyphenated patterns that matched contact-information patterns.

The results underline that deterministic post-generation filtering can be an effective means to provide an additional privacy layer, independent of the language model itself. The filter acts after the generation of AI responses and replaces contact information with the [hidden] placeholder prior to returning the processed response to the client.

Limitations

The evaluation corpus only includes a finite set of synthetic test patterns and cannot include every possible phone-number or email format, which would be available in the real world. Further testing with multilingual text, international numbering conventions, unusual formatting, and adversarial inputs should be performed before further security-related claims could be made.

5.2.6 Metric 6: Structured Output JSON Schema Conformity & Urgency Classification

Evaluation Objective

The objective of this experiment is to evaluate the output parser in the AI Provider Interface for conforming to the structured-output contract and assess urgency classification and downstream dispatch steering in the Conversational AI Assistant Module.

Experimental Setup

The study reports on 500 test cases (each evaluated 100 times) across five dimensions: format-resilience, service mapping, urgency classification, multi-turn slot filling, and downstream dispatch. Each dimension contained 100 test cases. The evaluation included four types of format resilience testing with raw JSON and markdown fenced blocks, response types from chatbot including plain text, and malformed-output cases. Service mapping involved multiple service domains. Two types of urgency cases were included with 50 emergency cases and 50 routine cases. Multi-turn slot filling covered Turns 1-4 of progressively revealed service, location, date, and time information. Finally, downstream dispatch was evaluated for both routing priority and normal routing cases.

The entire evaluation involved 50,000 iterations (500×100) of parser robustness, structured information extraction, conversational state management, and urgency propagation to downstream booking workflow.

Table 5.6: Structured Output Schema Conformity & Urgency Classification Across Dimensions

Evaluation Dimension	Test Cases	Successful Cases	Success Rate	Failure Cases	Mean Latency (µs)
Format Resilience (raw, fenced, plain, malformed)	100	100	100.0%	0	4.81
Service Mapping (multi-domain services)	100	100	100.0%	0	5.12
Urgency Classification (emergency vs. routine)	100	100	100.0%	0	5.43
Multi-turn Slot Filling (service → location → date → time)	100	100	100.0%	0	6.28
Downstream Dispatch (priority vs. normal routing)	100	100	100.0%	0	6.21
OVERALL	500	500	100.0%	0	Mean: 5.97 µs P99: 15.31 µs

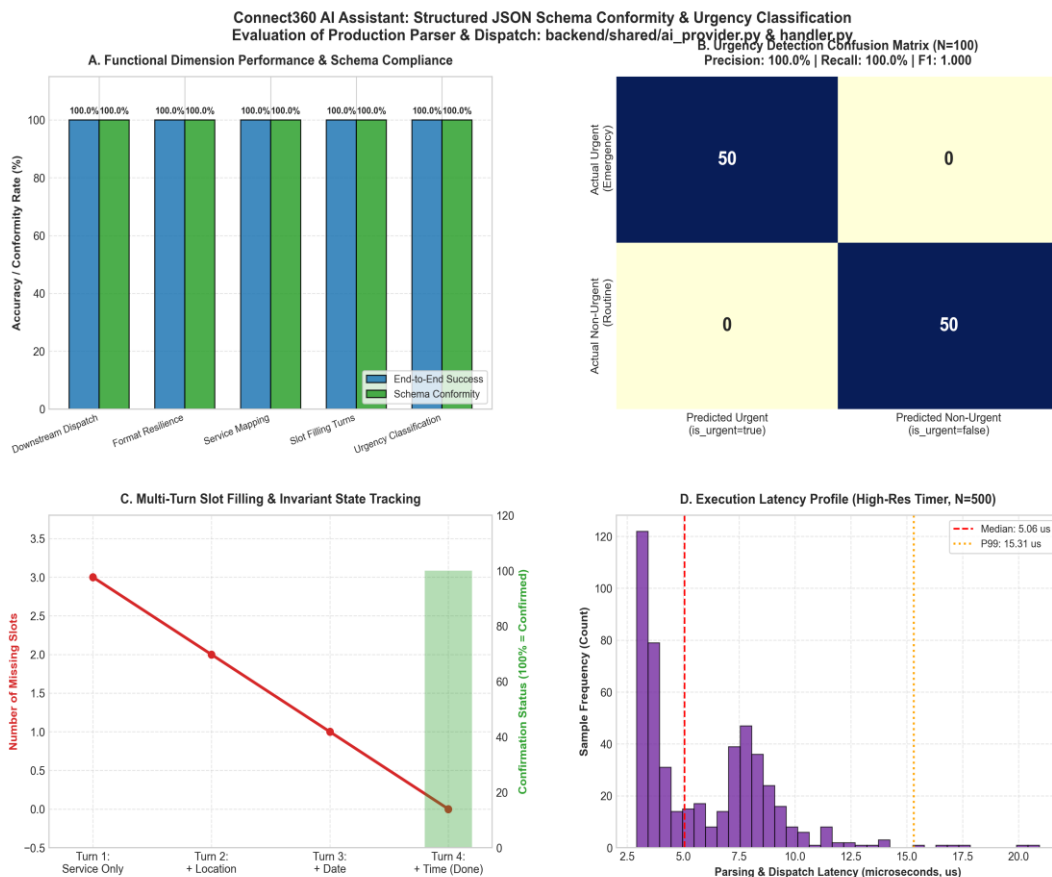


Figure 5.16: Structured Output JSON Schema Conformity & Urgency Classification

Discussion and Trends

Figure 5.16 shows that the structured-output parser maintained consistent schema conformity across the evaluated test cases. The evaluated dimensions showed consistent handling of format resilience, service mapping, urgency classification, multi-turn slot filling, and downstream dispatch. The urgency-classification evaluation showed correct classification for the tested emergency and routine cases, with no false-positive or false-negative classifications recorded within the evaluated cases. The system also maintains a confirmation invariant, preventing booking confirmation until the required information is obtained.

The multi-turn evaluation demonstrated progressive completion of booking information, with service, location, date, and time provided across successive turns before confirmation. The downstream dispatch process correctly propagated the evaluated urgency state to the booking workflow, directing urgent requests toward the Priority Booking path and non-urgent requests through normal candidate retrieval. The structured parsing and dispatch pipeline recorded a mean execution time of approximately 5.97 μ s, with a P99 latency of approximately 15.31 μ s in the reported benchmark. This measurement represents the local structured parsing and dispatch-processing stage and excludes external Gemini API and network latency.

Limitations

The structured-output evaluation was performed using a controlled test corpus and therefore does not cover every possible natural-language formulation, ambiguity, multilingual expression, or malformed response that may occur in production. Further evaluation with larger and more diverse conversational datasets would be required to assess generalisation beyond the tested cases.

5.3 OVERALL RESULT DISCUSSION

The experimental evaluation demonstrates the behavior of the major technical components of Connect360 under controlled test conditions. The Candidate Matching Engine exhibited approximately linear execution growth with increasing candidate-pool size, while Normal and Priority Booking resulted in different rankings via deliberate weight reallocation. The concurrency evaluation recorded no double-booking cases across 9,400 simulated concurrent requests, an indication that the conditional state-transition mechanism preserved booking consistency under the tested contention levels.

The spatial-routing evaluation observed that OSRM road distances were consistently greater than Haversine distances, with an overall weighted tortuosity factor of approximately 1.30. This finding supports the use of a lightweight geographic distance for candidate ranking and road-network routing after assignment. The contact-information redaction evaluation attained 99.20% recall with an overall F1-score of 0.9484, while the structured-output evaluation exhibited consistent schema conformity and successful handling across the evaluated test dimensions, with minimal parsing and dispatch overhead.

However, these results represent controlled experimental metrics rather than direct evidence of production-scale performance. Future evaluation should include larger real-world datasets, production-like cloud deployments, expanded geographic areas, and more diverse conversational inputs. The contact-information redaction mechanism should also be evaluated with additional multilingual, international, unusual, and adversarial input formats prior to making broader security-related claims.

CHAPTER 6

CONCLUSION AND FUTURE WORK

6.1 CONCLUSION

This Phase-I project designed, implemented and empirically evaluated Connect360, a serverless home-services marketplace that addresses worker matching, booking concurrency, spatial routing, conversational assistance and controlled contact-information handling. The implementation utilizes a three-tier architecture consisting of React 18, AWS Lambda, Amazon DynamoDB, Amazon Cognito, Amazon S3 and associated AI and routing services to bind natural-language customer interaction with multi-criteria worker allocation.

The empirical evaluation yielded the following major findings:

- **Matching Performance:** The multi-criteria matching engine demonstrated approximately linear execution growth with the candidate-pool size. For 100 candidate workers, the measured mean execution latency was 0.071 ms for Normal Booking and 0.076 ms for Priority Booking.
- **Rank Sensitivity:** A change in the weighting configuration between Normal and Priority Booking altered the candidate ordering. The evaluation recorded a mean absolute rank displacement of 13.18 positions, with a maximum displacement of 40 positions, to show that an increased emphasis on availability and distance changes worker ranking for priority requests.
- **Concurrency Consistency:** DynamoDB conditional and transactional operations maintained booking consistency during the concurrency evaluation. No double-booking cases were observed across 9,400 simulated concurrent requests under the tested conditions.

- **Spatial Routing:** The evaluated Chennai routes produced an overall weighted tortuosity factor of approximately 1.30, to show that road-network distances were generally greater than the corresponding Haversine distances. The results supported Haversine-based candidate ranking and OSRM-based road routing after assignment.
- **Contact-Information Redaction:** The deterministic contact-information redaction mechanism achieved 99.20% recall on the evaluated corpus, successfully intercepting 248 of 250 contact-information cases. The overall filtering F1-score was recorded as 0.9484.
- **Structured AI Processing:** The conversational AI pipeline displayed consistent structured-output handling across the evaluated test cases, with correct urgency classification recorded across the evaluated cases. Deterministic parsing and routing enabled urgent requests to be directed to the Priority Booking workflow and non-urgent requests to the appropriate worker recommendation workflow.

Overall, the Phase-I implementation demonstrated the integration of worker matching, transactional booking control, spatial routing, conversational assistance and controlled contact-information handling within a serverless home-services marketplace. The experimental results supported the implemented design under the tested conditions, with further validation required for larger real-world datasets, production-like deployments, broader geographic areas and more diverse conversational inputs.

6.2 FUTURE WORK

Building upon the Phase-I implementation and experimental findings, the following research and engineering enhancements are planned for Phase-II:

- **Dynamic Machine Learning Weight Tuning:** Replace the predefined Normal and Priority weighting configurations with adaptive ML or RL models that learn ranking weights from historical booking outcomes, worker responses, demand patterns and contextual information.
- **Dedicated OSRM Deployment:** Deploy a dedicated, containerized OSRM routing service within AWS to reduce dependence on external routing-server availability, and to improve consistency under higher request volumes.
- **Multi-Tier Urgency Classification:** Extend the current urgency classification into Low, Medium, High and Critical levels to provide more granular service prioritization and differentiated dispatch strategies.
- **Expanded Evaluation:** Extend the evaluation using larger and more diverse datasets, including additional contact-information formats, multilingual and international inputs, adversarial conversational cases, RBAC access scenarios, DynamoDB GSI performance, API and Lambda response characteristics, and complete end-to-end user-journey latency using automated testing frameworks.

REFERENCES

- [1] A. Ezhilnilavan, S. Gopikrishnan, V. Harish Aravindh, S. Mithun, and M. Saratha, "Local community service provider using machine learning," in Proc. 4th Int. Conf. Autom., Comput. Renewable Syst. (ICACRS), 2025, pp. 849–856, doi: 10.1109/ICACRS67045.2025.11324297.
- [2] H. Sun, F. Wang, Z. Tu, D. Sui, Y. Xing, K. Zhang, B. Zhao, and X. Xu, "A weighted preference optimization service recommendation method based on knowledge graph and large language model," in Proc. IEEE Int. Conf. Web Services (ICWS), 2025, doi: 10.1109/ICWS67624.2025.00014.
- [3] Y. Zhao, Z. Zhu, C. Gao, E. Wang, J. Huang, and F.-Y. Wang, "Heterogeneous graph reinforcement learning for dependency-aware multi-task allocation in spatial crowdsourcing," IEEE Trans. Mobile Comput., vol. 25, no. 3, pp. 3417–3434, Mar. 2026, doi: 10.1109/TMC.2025.3616027.
- [4] J. Chen, H. Miao, L. Jia, G. Yin, Y. Zhao, and K. Zheng, "Balancing competition for fairness-aware task recommendation and assignment in spatial crowdsourcing," in Proc. IEEE 42nd Int. Conf. Data Eng. (ICDE), 2026, pp. 2984–2996, doi: 10.1109/ICDE65706.2026.00222.
- [5] Q. Wu, Y. Li, J. Yan, M. Zhang, J. Xu, and M. Xu, "Adaptive task assignment in spatial crowdsourcing: A human-in-the-loop approach," IEEE Trans. Mobile Comput., vol. 24, no. 4, pp. 2726–2739, Apr. 2025, doi: 10.1109/TMC.2024.3501734.
- [6] S. Ge, S. Meng, J. Zhou, L. Qi, X. Xu, and X. Zhang, "LLMGCL: Graph contrastive learning with large language models for recommendation," in Proc. IEEE Int. Conf. Web Services (ICWS), 2025, doi: 10.1109/ICWS67624.2025.00016.
- [7] L. Deng, Y. Zhao, Y. Cui, Y. Xia, J. Chen, and K. Zheng, "Task recommendation in spatial crowdsourcing: A trade-off between diversity and coverage," in Proc. IEEE 40th Int. Conf. Data Eng. (ICDE), 2024, pp. 276–288.

- [8] Q. Wu, Y. Li, G. Zhu, B. Mei, J. Xu, and M. Xu, "Prediction-aware adaptive task assignment for spatial crowdsourcing," *IEEE Trans. Mobile Comput.*, vol. 23, no. 12, pp. 13048–13061, Dec. 2024, doi: 10.1109/TMC.2024.3423396.
- [9] Y. Zhao, X. Chen, G. Ye, F. Guo, K. Zheng, and X. Zhou, "Task allocation in spatial crowdsourcing: An efficient geographic partition framework," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 9, pp. 4943–4955, Sep. 2024, doi: 10.1109/TKDE.2024.3374086.
- [10] Y. Ma, X. Gao, S. S. Bhatti, and G. Chen, "Clustering based priority queue algorithm for spatial task assignment in crowdsourcing," *IEEE Trans. Services Comput.*, vol. 17, no. 2, pp. 452–465, Mar.–Apr. 2024, doi: 10.1109/TSC.2024.3353293.
- [11] A. Khelloufi, A. Khelil, A. Naouri, A. Ben Sada, H. Ning, N. Aung, and S. Dhelim, "A Hybrid Feature and Trust-Aggregation Recommender System in the Social Internet of Things," *IEEE Access*, vol. 12, pp. 126460–126477, 2024, doi: 10.1109/ACCESS.2024.3411887.
- [12] J. Chen, R. Yang, J. Guo, H. Wang, K. Wu, and L.-J. Zhang, "Reliable service recommendation: A multi-modal adversarial method for personalized recommendation under uncertain missing modalities," *IEEE Trans. Services Comput.*, vol. 18, no. 3, pp. 1724–1738, May–Jun. 2025, doi: 10.1109/TSC.2025.3556640.
- [13] Y. Munir, Q. Umer, M. Faheem, S. Akram, and A. Jaffar, "Developer recommendation and team formation in collaborative crowdsourcing platforms," *IEEE Access*, vol. 13, pp. 63170–63185, 2025, doi: 10.1109/ACCESS.2025.3558557.
- [14] Z. Zhao, W. Fan, J. Li, Y. Liu, X. Mei, Y. Wang, Z. Wen, F. Wang, X. Zhao, J. Tang, and Q. Li, "Recommender systems in the era of large language models (LLMs)," *IEEE Trans. Knowl. Data Eng.*, vol. 36, no. 11, pp. 6889–6907, Nov. 2024, doi: 10.1109/TKDE.2024.3392335.
- [15] B. Zheng, Y. Hou, H. Lu, Y. Chen, W. X. Zhao, M. Chen, and J.-R. Wen, "Adapting large language models by integrating collaborative semantics for recommendation," in

Proc. IEEE 40th Int. Conf. Data Eng. (ICDE), 2024, pp. 1435–1448, doi: 10.1109/ICDE60146.2024.00118.

[16] Refonaa, D. K. Yadav, G. Sood, S. K. Sonbhadra, and S. R. Reddy, "Exploring the potential of serverless architecture for cloud automation," in Proc. Int. Conf. Autom. Comput. (AUTOCOM), 2025, doi: 10.1109/AUTOCOM64127.2025.10957197.

[17] N. N. Ekwe-Ekwe and L. Amos, "The state of FaaS: An analysis of public functions-as-a-service providers," in Proc. IEEE 17th Int. Conf. Cloud Comput. (CLOUD), 2024, pp. 430–438, doi: 10.1109/CLOUD62652.2024.00055.

[18] M. Adeppady, A. Conte, P. Giaccone, H. Karl, and C. F. Chiasserini, "Dynamic management of constrained computing resources for serverless services," IEEE Trans. Netw. Service Manag., vol. 22, no. 2, pp. 1646–1663, Apr. 2025, doi: 10.1109/TNSM.2024.3497155.

[19] P. M. Kumar, P. Joshi, U. Dubey, A. Srivastava, and B. P. Kavin, "Enhanced job recommendation system using skill based rule matching algorithm," in Proc. Int. Conf. Data Sci. Bus. Syst. (ICDSBS), 2025, doi: 10.1109/ICDSBS63635.2025.11031850.

[20] V. R. Pasam, P. Devaraju, V. Methuku, K. Dharamshi, and S. M. Veerapaneni, "Engineering scalable AI pipelines: A cloud-native approach for intelligent transactional systems," in Proc. Int. Conf. Comput. Technol. (ICOCT), 2025, doi: 10.1109/ICOCT64433.2025.11118443.